

Jenkins CI Pipeline for Java Application with GitHub Integration

Author: Oluwaseun Osunsola

LinkedIn: <https://www.linkedin.com/in/oluwaseun-osunsola-95539b175/>

Environment & Tool: AWS, Docker, Docker Compose, Dockerfile

Project Link: <https://github.com/OluwaseunOa/DevOps-Projects/tree/main/Jenkins-Projects>

Project Overview

This comprehensive guide walks you through setting up Jenkins on an Ubuntu EC2 instance (AWS) for Continuous Integration (CI) of a simple Java application hosted on GitHub. Jenkins will automate the build process for a basic Java program (e.g., a "Hello World" style greeting app named `Test.java`).

By the end of this tutorial, you'll have:

- A running Jenkins server on AWS EC2.
- Security configurations for access.
- An Elastic IP for stable addressing.
- A Freestyle Jenkins job that clones a GitHub repository, compiles, and runs a Java file.
- Hands-on experience with CI basics.

This setup assumes a basic understanding of AWS, Linux commands, and GitHub. The process is illustrated with screenshots for clarity.

Prerequisites

- An AWS account with an Ubuntu EC2 instance launched (t2.micro is sufficient for testing).
- SSH key pair for accessing the EC2 instance.
- A GitHub account with a repository containing a simple Java file (e.g., `Test.java` with code like

```
public class Test { public static void main(String[] args) { System.out.println("Greeting!"); } }.
```
- Basic tools: Web browser, terminal/SSH client.

Important Notes

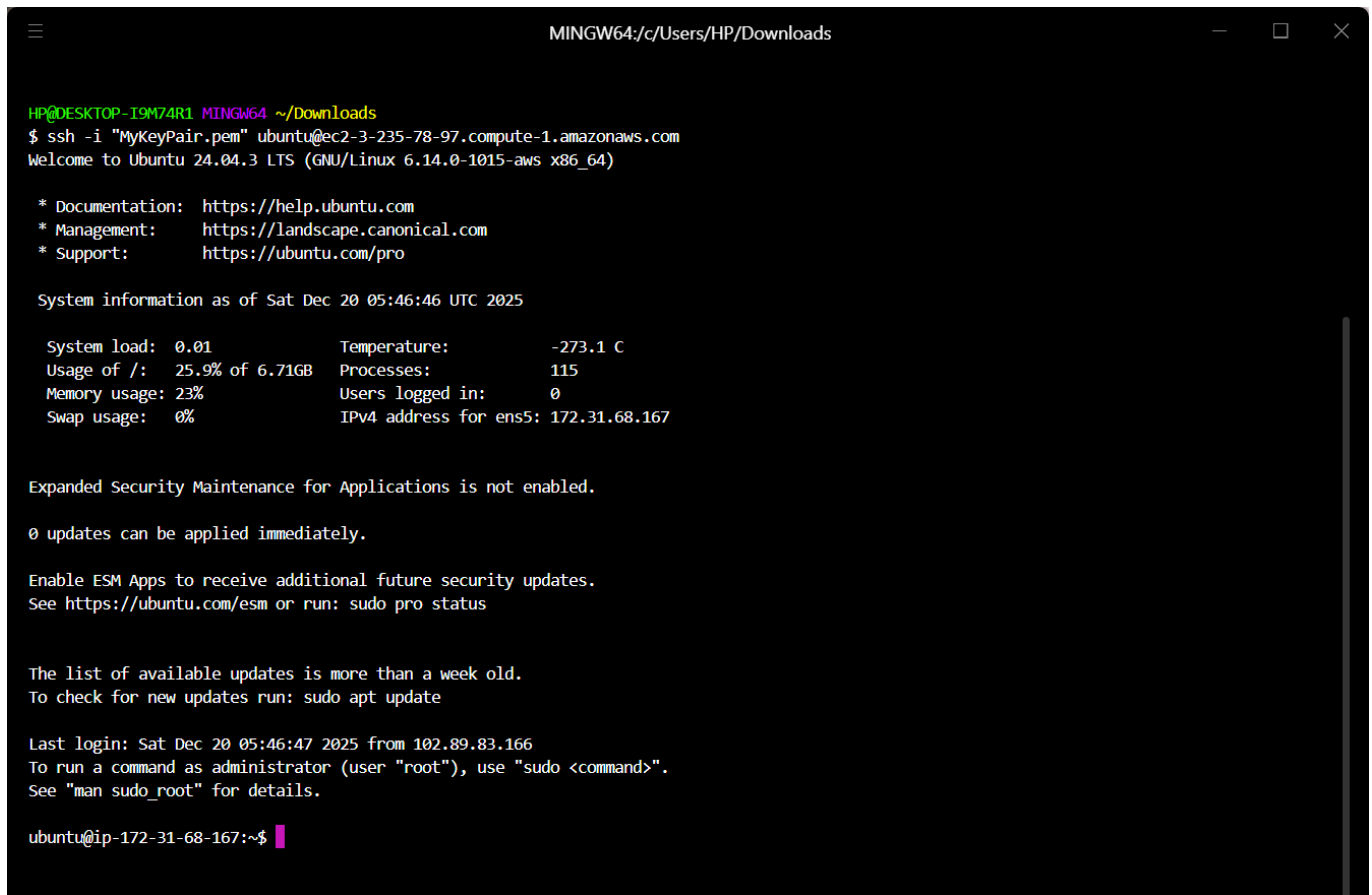
- All commands are run on the Ubuntu EC2 instance unless specified.
- Replace placeholders like `<your-public-ip>` or repository URLs with your own.
- This guide uses OpenJDK 21 (compatible with Jenkins as of December 2025).
- Security: Use strong passwords; restrict firewall rules in production.
- Costs: EC2 and Elastic IP may incur minor AWS charges.

Step-by-Step Setup

Step 1: SSH into Your Ubuntu EC2 Server

Connect to your EC2 instance via SSH to begin the setup.

- Command: `ssh -i your-key.pem ubuntu@<ec2-public-ip>`
- Ensure you're logged in as the ubuntu user with sudo privileges.

A terminal window titled 'MINGW64:/c/Users/HP/Downloads' showing an SSH session. The user 'HP@DESKTOP-I9M74R1' connects to 'ubuntu@ec2-3-235-78-97.compute-1.amazonaws.com'. The terminal displays the Ubuntu 24.04.3 LTS welcome message, system information, and security notices. The prompt is 'ubuntu@ip-172-31-68-167:~\$'.

```
HP@DESKTOP-I9M74R1 MINGW64 ~/Downloads
$ ssh -i "MyKeyPair.pem" ubuntu@ec2-3-235-78-97.compute-1.amazonaws.com
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1015-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sat Dec 20 05:46:46 UTC 2025

System load:  0.01          Temperature:   -273.1 C
Usage of /:   25.9% of 6.71GB Processes:      115
Memory usage: 23%          Users logged in: 0
Swap usage:   0%           IPv4 address for ens5: 172.31.68.167

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

Last login: Sat Dec 20 05:46:47 2025 from 102.89.83.166
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-172-31-68-167:~$
```

Step 2: Update and Upgrade the System

Keep your system packages up to date for security and compatibility.

- Command: `sudo apt update && sudo apt upgrade -y`

```

ubuntu@ip-172-31-68-167:~$ sudo apt update && sudo apt upgrade -y
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:5 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 Packages [15.0 MB]
Get:6 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe Translation-en [5982 kB]
Get:7 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 Components [3871 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1391 kB]
Get:9 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 c-n-f Metadata [301 kB]
Get:10 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse amd64 Packages [269 kB]
Get:11 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse Translation-en [118 kB]
Get:12 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse amd64 Components [35.0 kB]
Get:13 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse amd64 c-n-f Metadata [8328 B]
Get:14 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1684 kB]
Get:15 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main Translation-en [311 kB]
Get:16 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [175 kB]
Get:17 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 c-n-f Metadata [15.8 kB]
Get:18 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1506 kB]
Get:19 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [225 kB]
Get:20 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.6 kB]
Get:21 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [9504 B]
Get:22 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [916 kB]
Get:23 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [207 kB]
Get:24 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe Translation-en [306 kB]
Get:25 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [71.5 kB]
Get:26 http://security.ubuntu.com/ubuntu noble-security/universe amd64 c-n-f Metadata [19.4 kB]
Get:27 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [2286 kB]
Get:28 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [377 kB]
Get:29 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 c-n-f Metadata [31.4 kB]
Get:30 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [2413 kB]
Get:31 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/restricted Translation-en [550 kB]
Get:32 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Components [212 B]
Get:33 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 c-n-f Metadata [516 B]
Get:34 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Packages [30.3 kB]
Get:35 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/multiverse Translation-en [6048 B]
Get:36 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:37 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 c-n-f Metadata [488 B]
Get:38 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-backports/main amd64 Packages [64.0 kB]

```

Step 3: Check if Git is Installed

Verify Git is available (required for repository integration later).

- Command: `git --version`
- If not installed, run `sudo apt install git -y`.

```

MINGW64/c/Users/HP/Downloads
ubuntu@ip-172-31-68-167:~$ git --version
git version 2.43.0
ubuntu@ip-172-31-68-167:~$

```

Step 4: Install OpenJDK 21 JRE

Jenkins requires Java. Install OpenJDK 21 for the latest compatibility.

- Command: `sudo apt install openjdk-21-jre -y`

```

MINGW64:/c/Users/HP/Downloads

ubuntu@ip-172-31-68-167:~$ sudo apt install openjdk-21-jre -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  adwaita-icon-theme alsa-topology-conf alsa-ucm-conf at-spi2-common at-spi2-core ca-certificates-java
  dconf-gsettings-backend dconf-service fontconfig fontconfig-config fonts-dejavu-core fonts-dejavu-extra fonts-dejavu-mono
  gsettings-desktop-schemas gtk-update-icon-cache hicolor-icon-theme humanity-icon-theme java-common libasound2-data
  libasound2t64 libatk-bridge2.0-0t64 libatk-wrapper-java libatk-wrapper-java-jni libatk1.0-0t64 libatspi2.0-0t64
  libavahi-client3 libavahi-common-data libavahi-common3 libcairo-gobject2 libcairo2 libcolord2 libcups2t64 libdatrie1
  libdconf1 libdeflate0 libdrm-amdgpu1 libdrm-intel1 libepoxy0 libfontconfig1 libgbm1 libgdk-pixbuf-2.0-0
  libgdk-pixbuf2.0-bin libgdk-pixbuf2.0-common libgif7 libgl1 libgl1-mesa-dri libglvnd0 libglx-mesa0 libglx0 libgraphite2-3
  libgtk-3-0t64 libgtk-3-bin libgtk-3-common libharfbuzz0b libice6 libjbig0 libjpeg-turbo8 libjpeg8 liblcms2-2 liblerc4
  libllvmlt20 libpango-1.0-0 libpangocairo-1.0-0 libpangoft2-1.0-0 libpciaccess0 libpcsclite1 libpixman-1-0 librsvg2-2
  librsvg2-common libsharpyuv0 libsm6 libthai-data libthai0 libtiff6 libvulkan1 libwayland-client0 libwayland-cursor0
  libwayland-egl1 libwayland-server0 libwebp7 libx11-xcb1 libxaw7 libxcb-dri3-0 libxcb-glx0 libxcb-present0 libxcb-randr0
  libxcb-render0 libxcb-shape0 libxcb-shm0 libxcb-sync1 libxcb-xfixes0 libxcomposite1 libxcursor1 libxdamage1 libxf86vm3
  libxft2 libxi6 libxinerama1 libxkbfile1 libxmu6 libxpm4 libxrandr2 libxrender1 libxshmfence1 libxt6t64 libxtst6 libxv1
  libxxf86dga1 libxxf86vm1 mesa-libgallium mesa-vulkan-drivers openjdk-21-jre-headless session-migration ubuntu-mono
  x11-common x11-utils
Suggested packages:
  default-jre alsa-utils libasound2-plugins colord cups-common gvfs liblcms2-utils pcsclib librsvg2-bin libnss-mdns
  fonts-ipafont-gothic fonts-ipafont-mincho fonts-wqy-microhei | fonts-wqy-zenhei fonts-indic mesa-utils
Recommended packages:
  luit
The following NEW packages will be installed:
  adwaita-icon-theme alsa-topology-conf alsa-ucm-conf at-spi2-common at-spi2-core ca-certificates-java
  dconf-gsettings-backend dconf-service fontconfig fontconfig-config fonts-dejavu-core fonts-dejavu-extra fonts-dejavu-mono
  gsettings-desktop-schemas gtk-update-icon-cache hicolor-icon-theme humanity-icon-theme java-common libasound2-data
  libasound2t64 libatk-bridge2.0-0t64 libatk-wrapper-java libatk-wrapper-java-jni libatk1.0-0t64 libatspi2.0-0t64
  libavahi-client3 libavahi-common-data libavahi-common3 libcairo-gobject2 libcairo2 libcolord2 libcups2t64 libdatrie1
  libdconf1 libdeflate0 libdrm-amdgpu1 libdrm-intel1 libepoxy0 libfontconfig1 libgbm1 libgdk-pixbuf-2.0-0
  libgdk-pixbuf2.0-bin libgdk-pixbuf2.0-common libgif7 libgl1 libgl1-mesa-dri libglvnd0 libglx-mesa0 libglx0 libgraphite2-3
  libgtk-3-0t64 libgtk-3-bin libgtk-3-common libharfbuzz0b libice6 libjbig0 libjpeg-turbo8 libjpeg8 liblcms2-2 liblerc4
  libllvmlt20 libpango-1.0-0 libpangocairo-1.0-0 libpangoft2-1.0-0 libpciaccess0 libpcsclite1 libpixman-1-0 librsvg2-2
  librsvg2-common libsharpyuv0 libsm6 libthai-data libthai0 libtiff6 libvulkan1 libwayland-client0 libwayland-cursor0
  libwayland-egl1 libwayland-server0 libwebp7 libx11-xcb1 libxaw7 libxcb-dri3-0 libxcb-glx0 libxcb-present0 libxcb-randr0
  libxcb-render0 libxcb-shape0 libxcb-shm0 libxcb-sync1 libxcb-xfixes0 libxcomposite1 libxcursor1 libxdamage1 libxf86vm3
  libxft2 libxi6 libxinerama1 libxkbfile1 libxmu6 libxpm4 libxrandr2 libxrender1 libxshmfence1 libxt6t64 libxtst6 libxv1

```

Step 5: Verify Java Installation

Confirm Java is installed correctly.

- Command: `java -version`
- Look for output like "openjdk 21.x".

```

MINGW64:/c/Users/HP/Downloads

ubuntu@ip-172-31-68-167:~$ java --version
openjdk 21.0.9 2025-10-21
OpenJDK Runtime Environment (build 21.0.9+10-Ubuntu-124.04)
OpenJDK 64-Bit Server VM (build 21.0.9+10-Ubuntu-124.04, mixed mode, sharing)
ubuntu@ip-172-31-68-167:~$

```

Step 6: Download Jenkins Repository Key

Add the Jenkins GPG key for secure package installation.

- Command: `sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc`
`https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key`

```
MINGW64:/c/Users/HP/Downloads

ubuntu@ip-172-31-68-167:~$ sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
--2025-12-20 06:01:48-- https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
Resolving pkg.jenkins.io (pkg.jenkins.io)... 146.75.38.133, 2a04:4e42:78::645
Connecting to pkg.jenkins.io (pkg.jenkins.io)[146.75.38.133]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3175 (3.1K) [application/pgp-keys]
Saving to: '/etc/apt/keyrings/jenkins-keyring.asc'

/etc/apt/keyrings/jenkins-keyri 100%[=====>] 3.10K --.-KB/s in 0s

2025-12-20 06:01:48 (47.9 MB/s) - '/etc/apt/keyrings/jenkins-keyring.asc' saved [3175/3175]

ubuntu@ip-172-31-68-167:~$
```

Step 7: Add Jenkins Repository to APT Sources

Register the Jenkins repository.

- Command: `echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian-stable binary/" | sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null`

```
MINGW64:/c/Users/HP/Downloads

ubuntu@ip-172-31-68-167:~$ echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian-stable binary/" | sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null
ubuntu@ip-172-31-68-167:~$
```

Step 8: Update Package List

Refresh the package index to include Jenkins.

- Command: `sudo apt update`

```
MINGW64:/c/Users/HP/Downloads

ubuntu@ip-172-31-68-167:~$ sudo apt update
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease
Ign:4 https://pkg.jenkins.io/debian-stable binary/ InRelease
Get:5 https://pkg.jenkins.io/debian-stable binary/ Release [2044 B]
Get:6 https://pkg.jenkins.io/debian-stable binary/ Release.gpg [833 B]
Hit:7 http://security.ubuntu.com/ubuntu noble-security InRelease
Get:8 https://pkg.jenkins.io/debian-stable binary/ Packages [30.3 kB]
Fetched 33.2 kB in 0s (86.0 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
ubuntu@ip-172-31-68-167:~$
```

Step 9: Install Jenkins

Install the Jenkins package.

- Command: `sudo apt install jenkins -y`

```
ubuntu@ip-172-31-68-167:~$ sudo apt install jenkins -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  net-tools
The following NEW packages will be installed:
  jenkins net-tools
0 upgraded, 2 newly installed, 0 to remove and 0 not upgraded.
Need to get 95.2 MB of archives.
After this operation, 96.3 MB of additional disk space will be used.
Get:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 net-tools amd64 2.10-0.1ubuntu4.4 [204 kB]
Get:2 https://pkg.jenkins.io/debian-stable binary/ jenkins 2.528.3 [95.0 MB]
Fetched 95.2 MB in 1s (64.4 MB/s)
```

Step 10: Enable and Start Jenkins

Start the Jenkins service and enable it on boot.

- Command: `sudo systemctl enable jenkins && sudo systemctl start jenkins`

```
ubuntu@ip-172-31-68-167:~$ sudo systemctl enable --now jenkins
Synchronizing state of jenkins.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable jenkins
ubuntu@ip-172-31-68-167:~$
```

Step 11: Check Jenkins Service Status

Verify Jenkins is running.

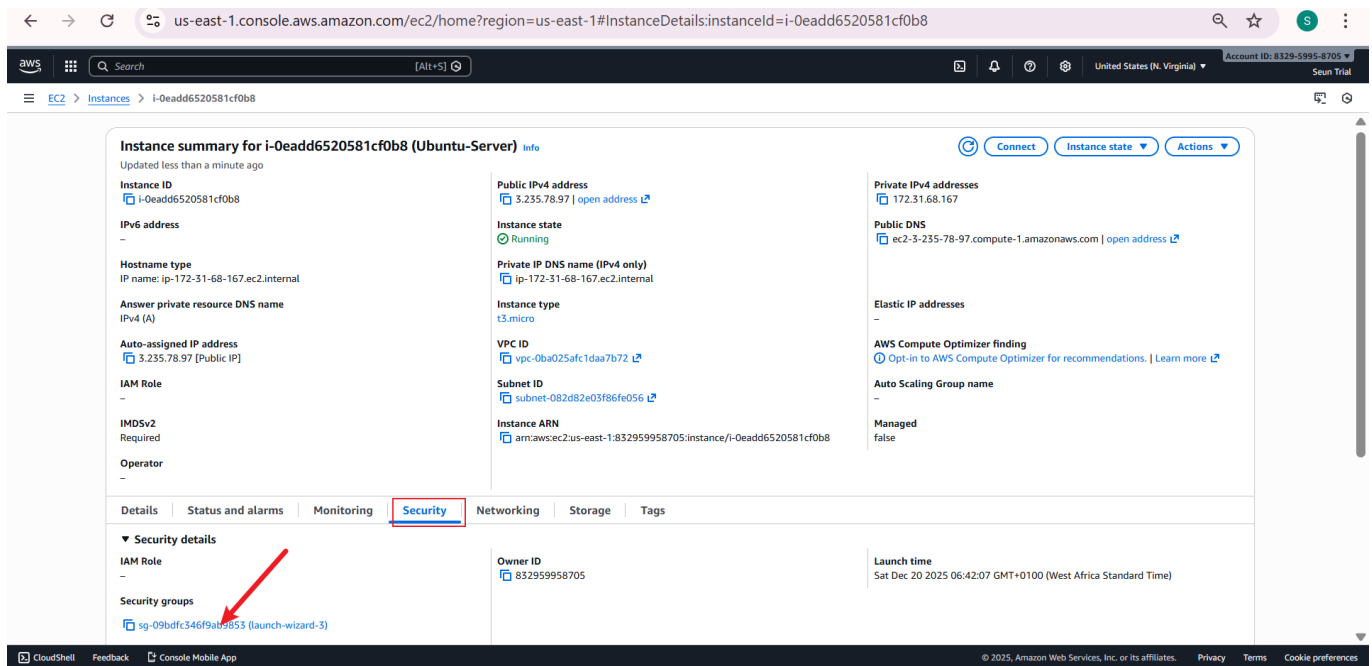
- Command: `sudo systemctl status jenkins`
- Look for "Active: active (running)".

```
ubuntu@ip-172-31-68-167:~$ sudo systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)
   Active: active (running) since Sat 2025-12-20 06:07:18 UTC; 1min 35s ago
     Main PID: 15292 (java)
        Tasks: 37 (limit: 1008)
      Memory: 304.0M (peak: 319.6M)
         CPU: 18.381s
    CGroup: /system.slice/jenkins.service
            └─15292 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war
Dec 20 06:07:14 ip-172-31-68-167 jenkins[15292]: [LF]> This may also be found at: /var/lib/jenkins/secrets/initialAdminPassword
Dec 20 06:07:14 ip-172-31-68-167 jenkins[15292]: [LF]>
Dec 20 06:07:14 ip-172-31-68-167 jenkins[15292]: [LF]> *****
Dec 20 06:07:14 ip-172-31-68-167 jenkins[15292]: [LF]> *****
Dec 20 06:07:14 ip-172-31-68-167 jenkins[15292]: [LF]> *****
Dec 20 06:07:18 ip-172-31-68-167 jenkins[15292]: 2025-12-20 06:07:18.147+0000 [id=31] INFO jenkins.InitReactorRu
Dec 20 06:07:18 ip-172-31-68-167 jenkins[15292]: 2025-12-20 06:07:18.188+0000 [id=24] INFO hudson.lifecycle.Life
Dec 20 06:07:18 ip-172-31-68-167 systemd[1]: Started jenkins.service - Jenkins Continuous Integration Server.
Dec 20 06:07:18 ip-172-31-68-167 jenkins[15292]: 2025-12-20 06:07:18.490+0000 [id=50] INFO h.m.DownloadService$D
Dec 20 06:07:18 ip-172-31-68-167 jenkins[15292]: 2025-12-20 06:07:18.492+0000 [id=50] INFO hudson.util.Retrier#s
ubuntu@ip-172-31-68-167:~$
```

Step 12: Navigate to Security Groups in AWS Console

In the AWS EC2 dashboard, configure inbound rules for port 8080.

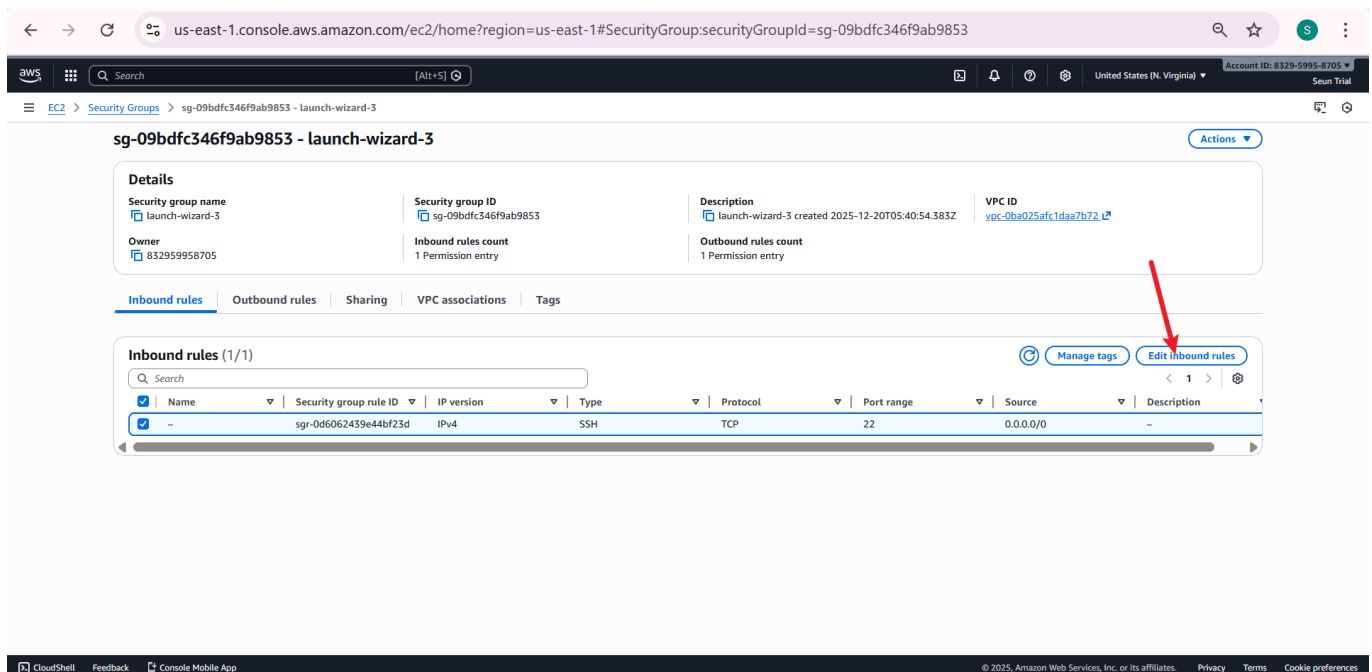
- Go to your instance > Security tab > Click on the Security Group link.



Step 13: Edit Inbound Rules

Modify the security group to allow access.

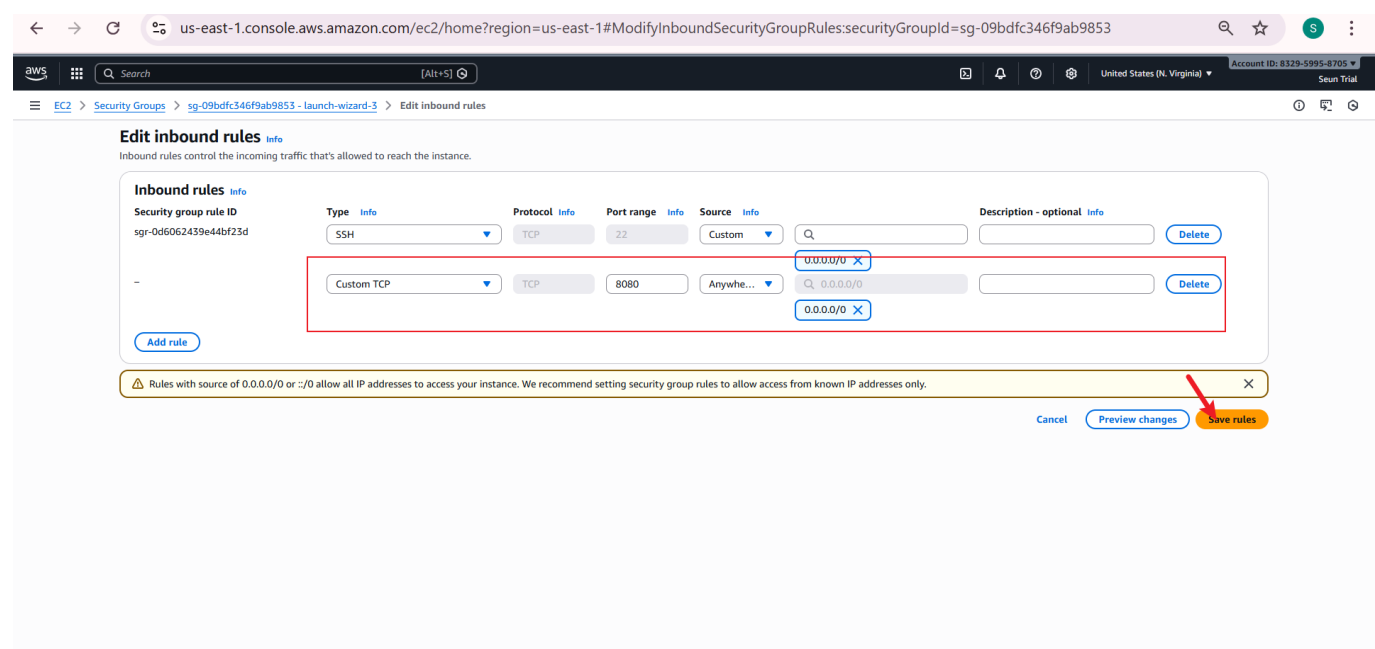
- Click "Edit inbound rules".



Step 14: Add Rule for Port 8080

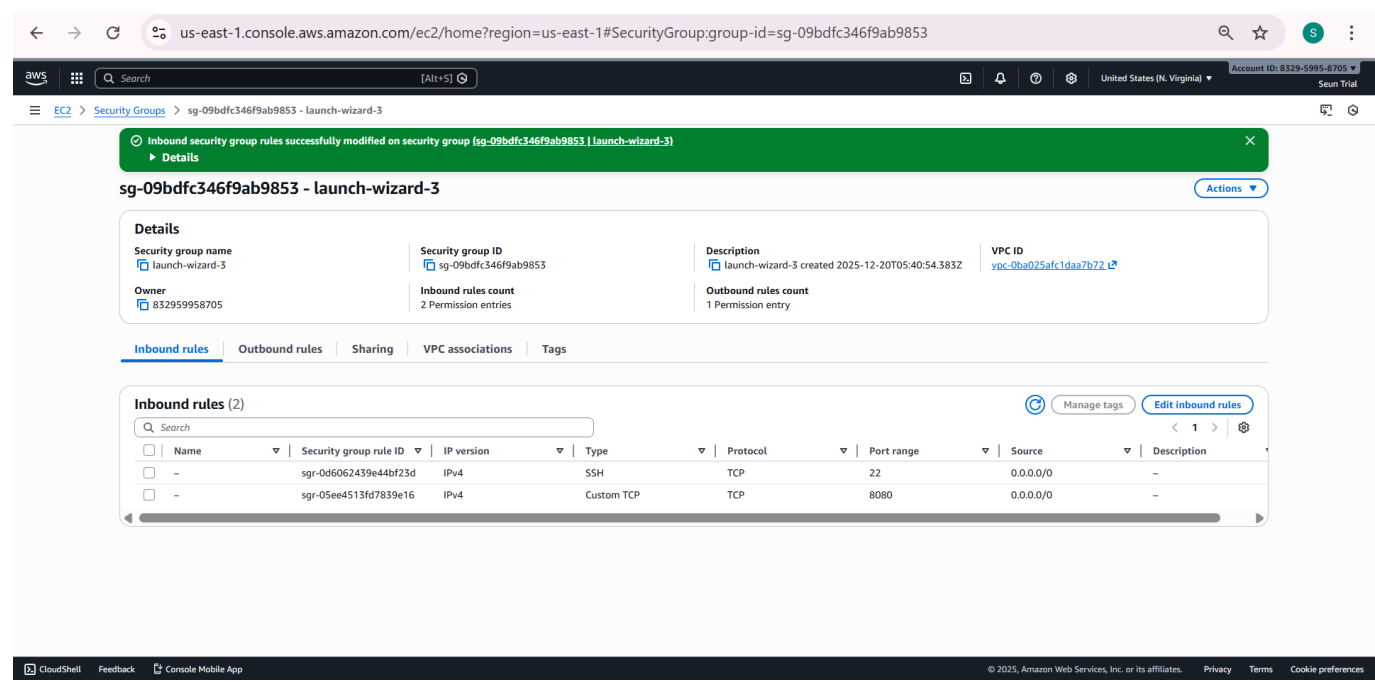
Allow TCP traffic on port 8080 from anywhere (0.0.0.0/0 for IPv4). For production, restrict to your IP.

- Type: Custom TCP, Port: 8080, Source: Anywhere-IPv4.



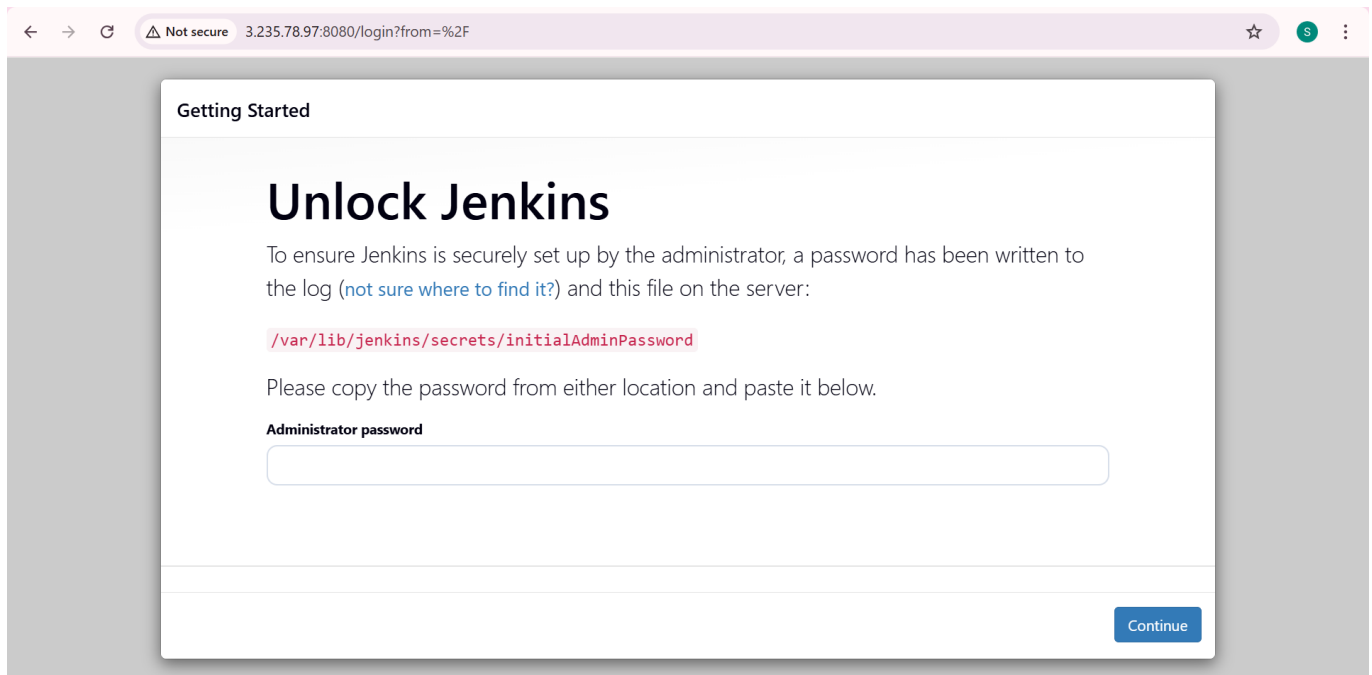
Step 15: Confirm Security Group Modification

Save the changes and verify the update.



Step 16: Access Jenkins Web Interface

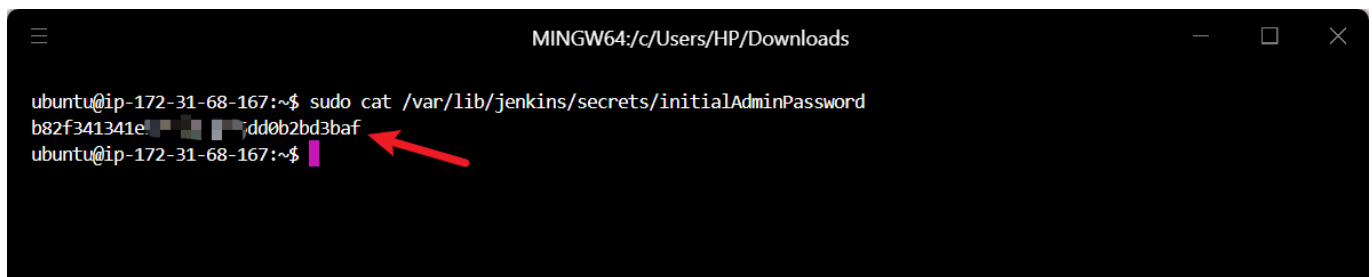
Open a browser and visit `http://<ec2-public-ip>:8080`. The "Getting Started" page should appear.



Step 17: Retrieve Initial Admin Password

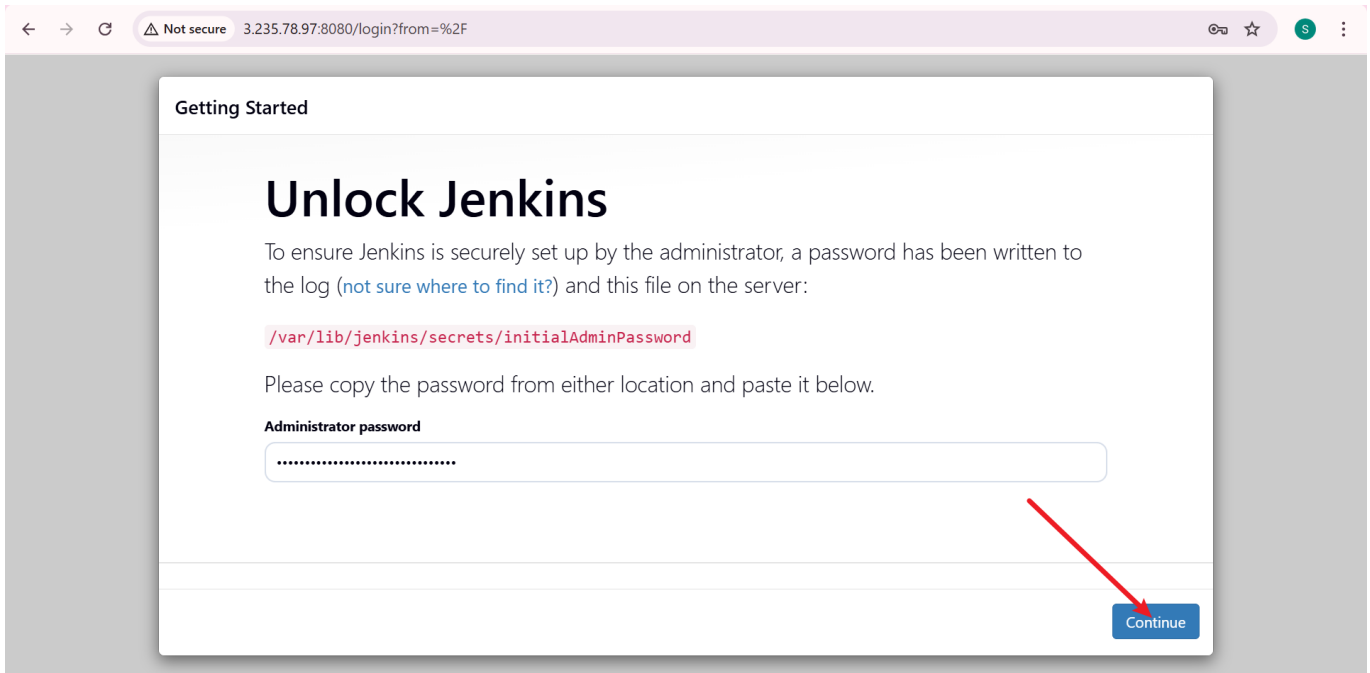
Get the unlock password from the server.

- Command: `sudo cat /var/lib/jenkins/secrets/initialAdminPassword`



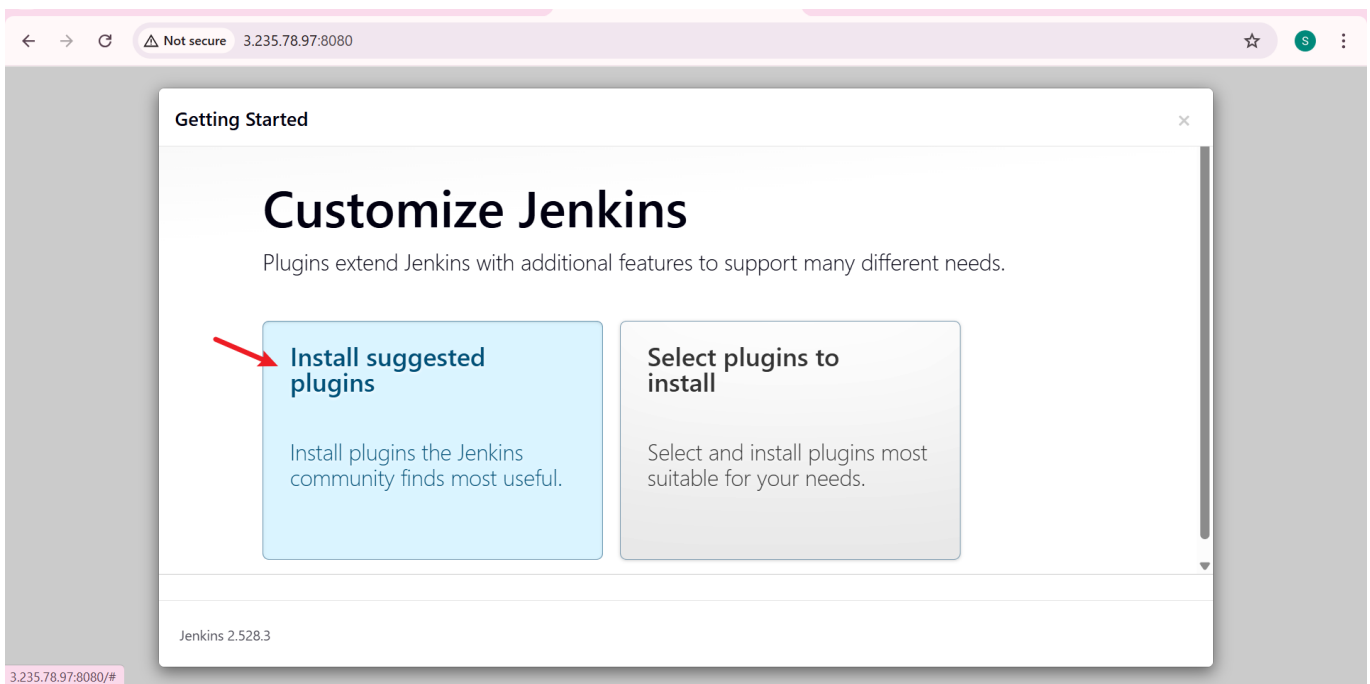
Step 18: Unlock Jenkins

Paste the password on the web page and continue.



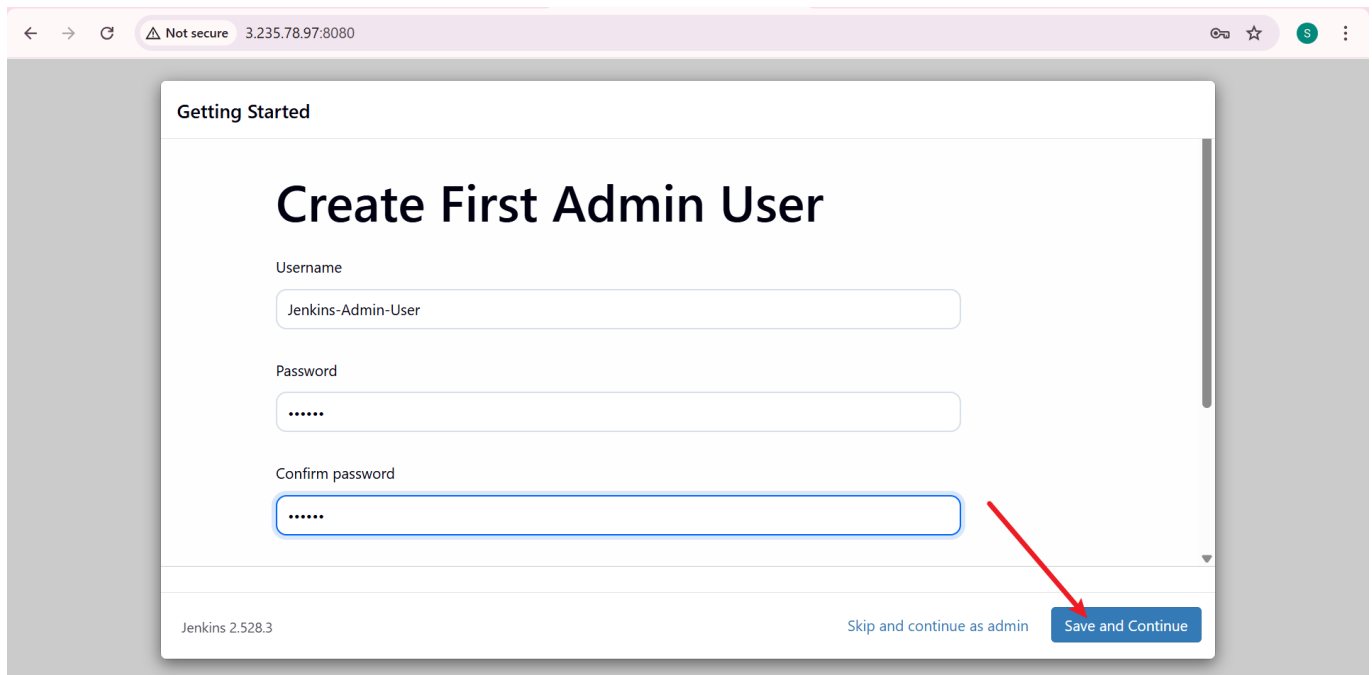
Step 19: Install Suggested Plugins

Choose "Install suggested plugins" for essentials like Git integration.



Step 20: Create First Admin User

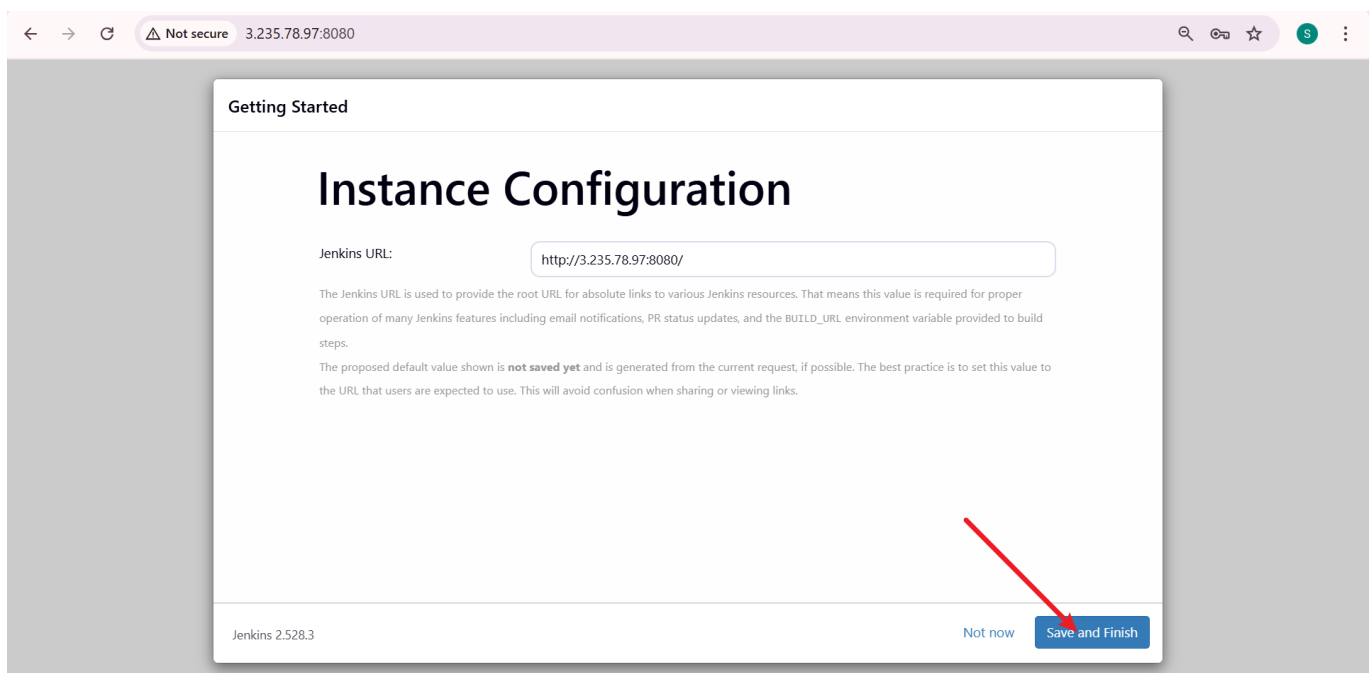
Set up credentials: Username, password, full name, email.



The screenshot shows the 'Getting Started' page of Jenkins 2.528.3. The main heading is 'Create First Admin User'. There are three input fields: 'Username' with the value 'Jenkins-Admin-User', 'Password' with masked characters '*****', and 'Confirm password' also with masked characters '*****'. A red arrow points to the 'Save and Continue' button at the bottom right. At the bottom left, it says 'Jenkins 2.528.3'. At the bottom center, there is a link 'Skip and continue as admin'.

Step 21: Confirm Jenkins URL

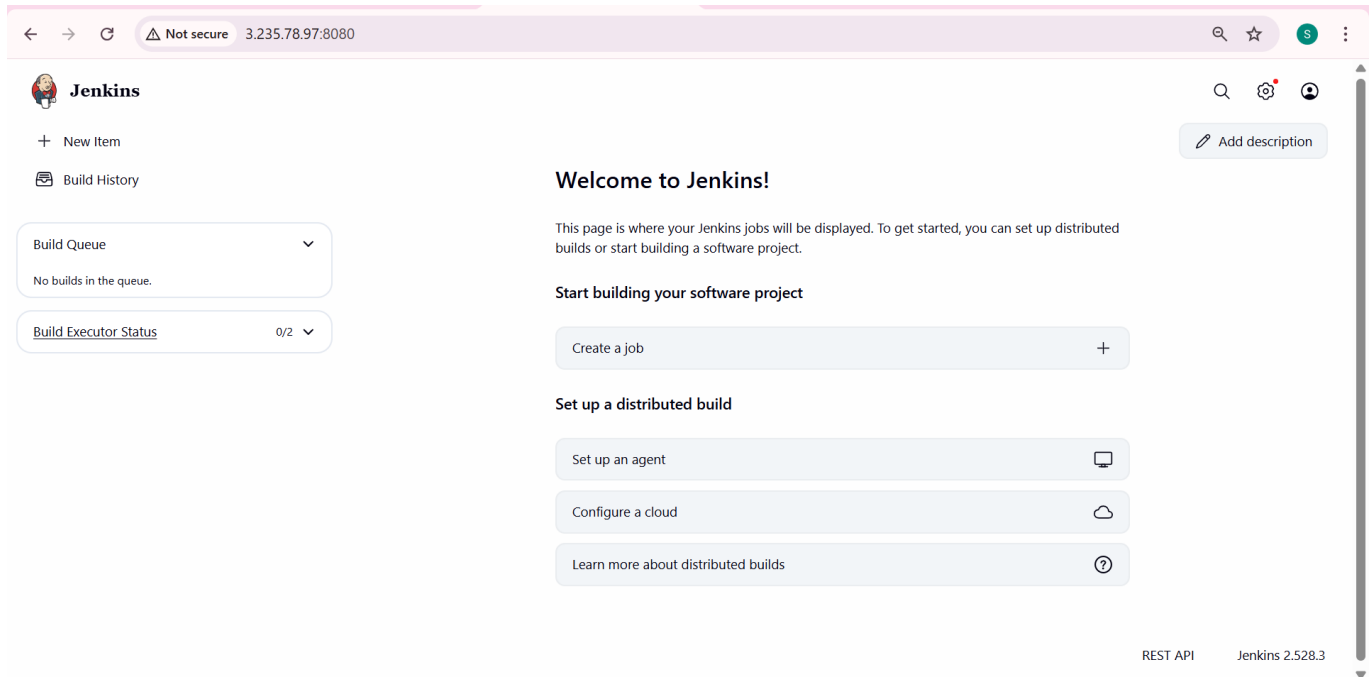
Leave it at the default for now (we'll update later with Elastic IP).



The screenshot shows the 'Getting Started' page of Jenkins 2.528.3. The main heading is 'Instance Configuration'. There is a 'Jenkins URL:' label followed by a text input field containing 'http://3.235.78.97:8080/'. Below the input field, there is explanatory text: 'The Jenkins URL is used to provide the root URL for absolute links to various Jenkins resources. That means this value is required for proper operation of many Jenkins features including email notifications, PR status updates, and the BUILD_URL environment variable provided to build steps. The proposed default value shown is **not saved yet** and is generated from the current request, if possible. The best practice is to set this value to the URL that users are expected to use. This will avoid confusion when sharing or viewing links.' A red arrow points to the 'Save and Finish' button at the bottom right. At the bottom left, it says 'Jenkins 2.528.3'. At the bottom center, there is a link 'Not now'.

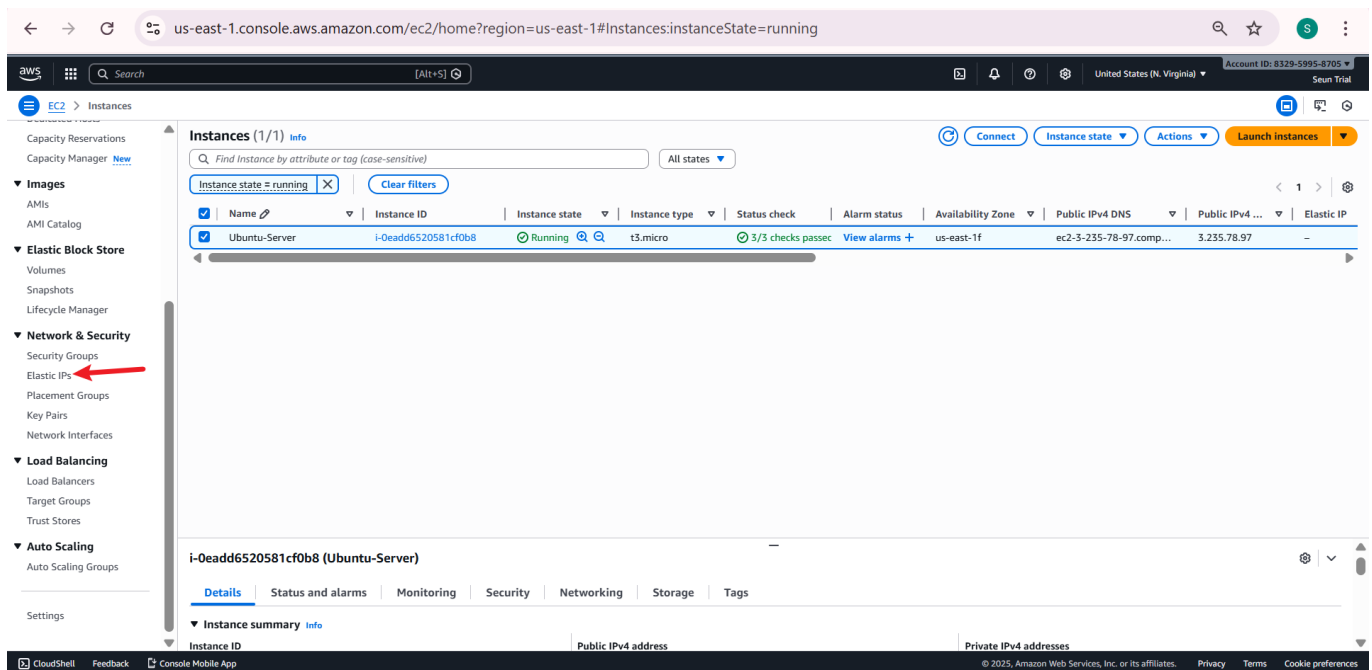
Step 22: Jenkins Dashboard

You're now on the main dashboard.



Step 23: Navigate to Elastic IPs in AWS Console

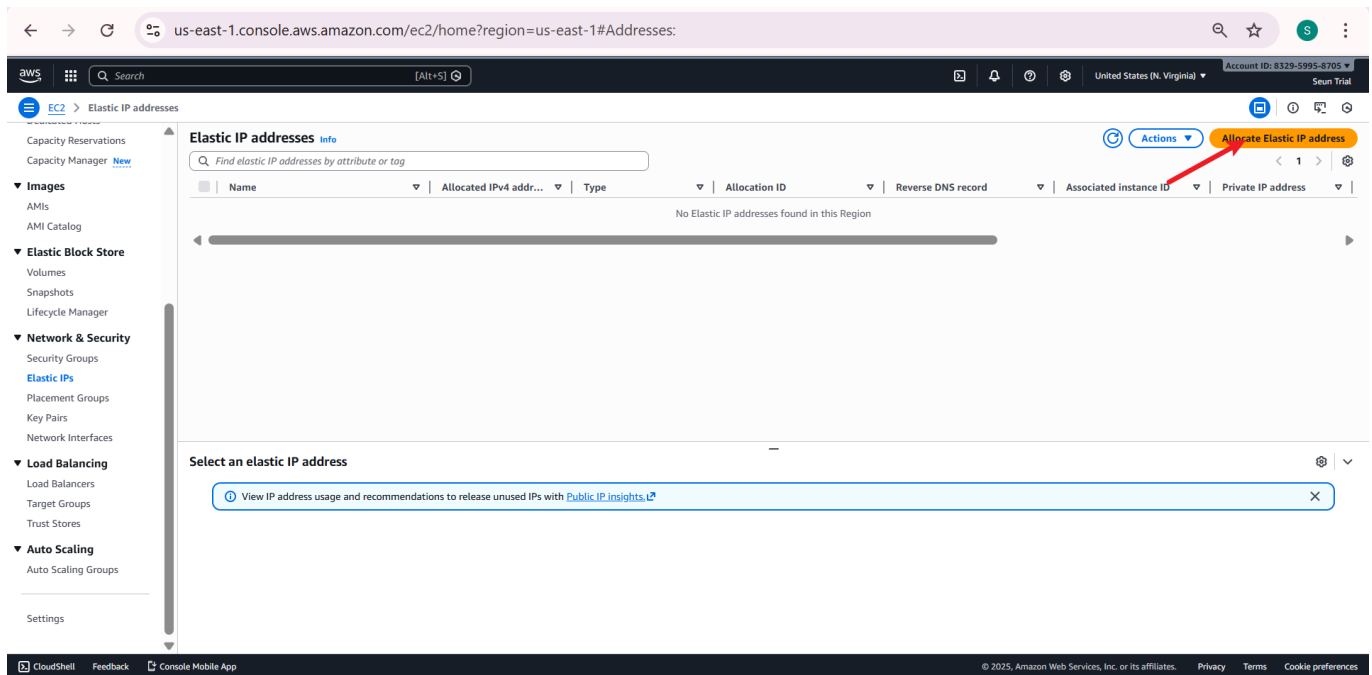
For a static IP, go to EC2 > Network & Security > Elastic IPs.



Step 24: Allocate Elastic IP

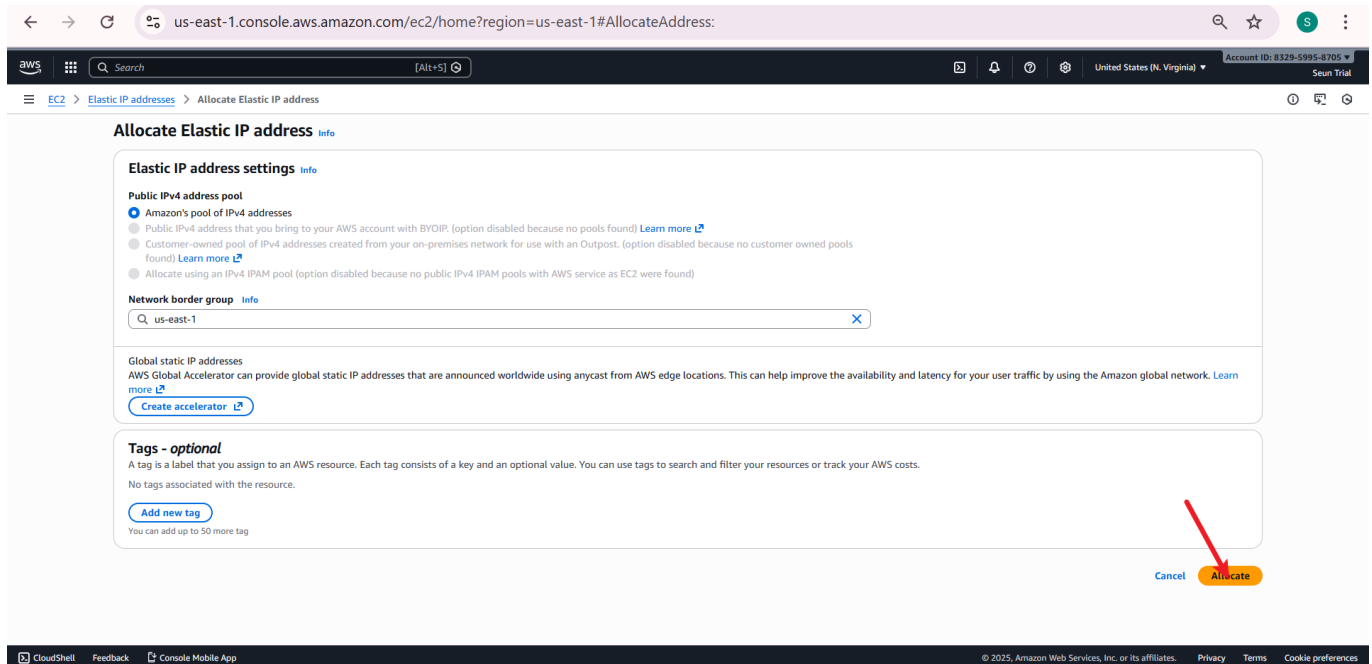
Create a new static IP.

- Click "Allocate Elastic IP address".



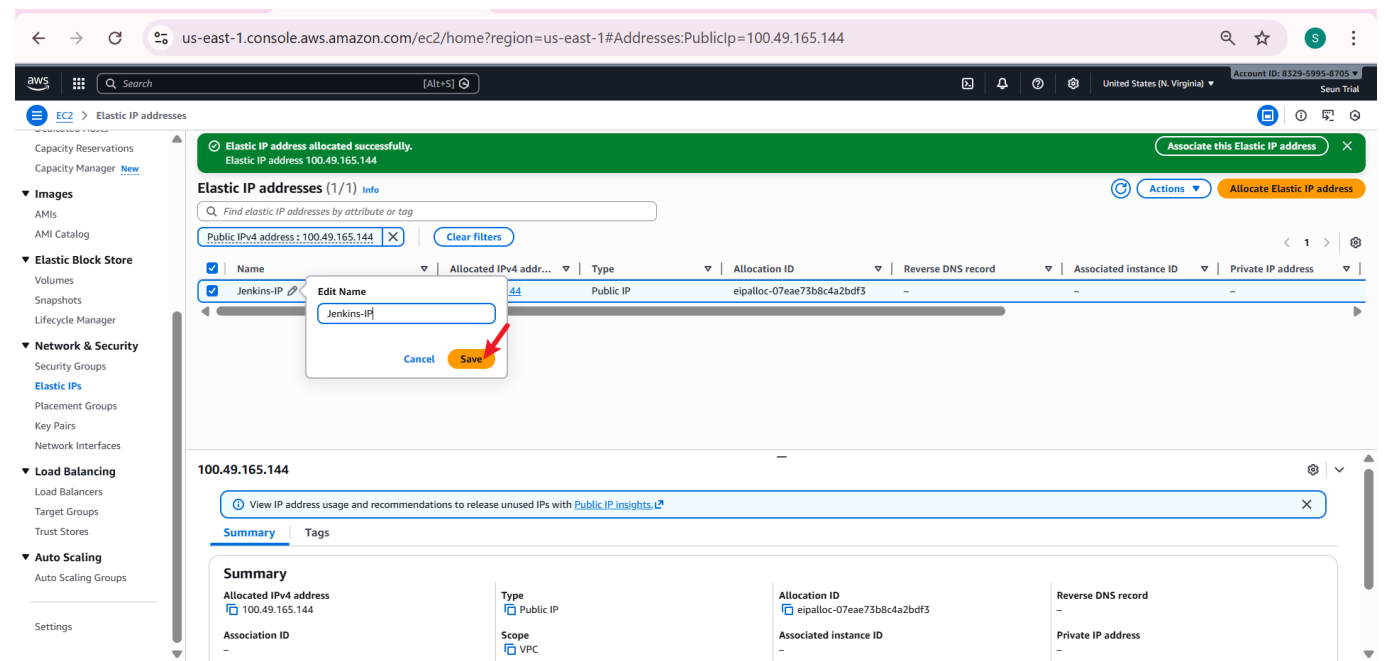
Step 25: Confirm Allocation

Proceed with allocation (use Amazon's pool).



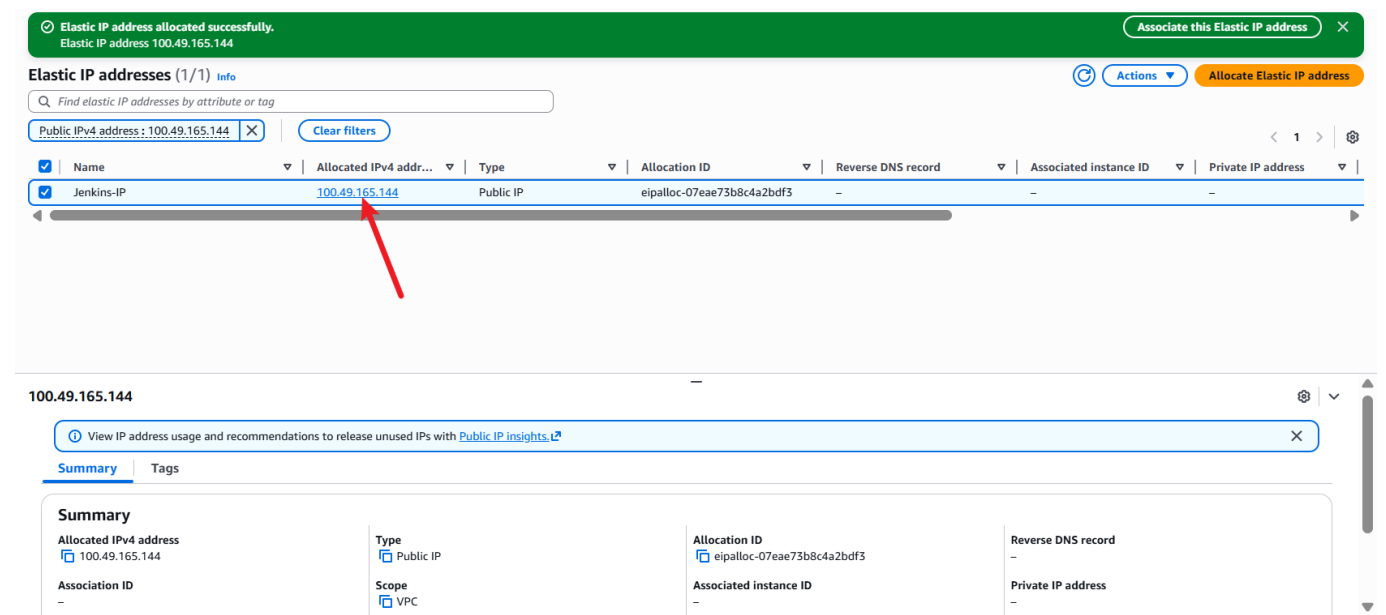
Step 26: Name the Elastic IP

Tag it as "jenkins-ip" for identification.



Step 27: Select the IP

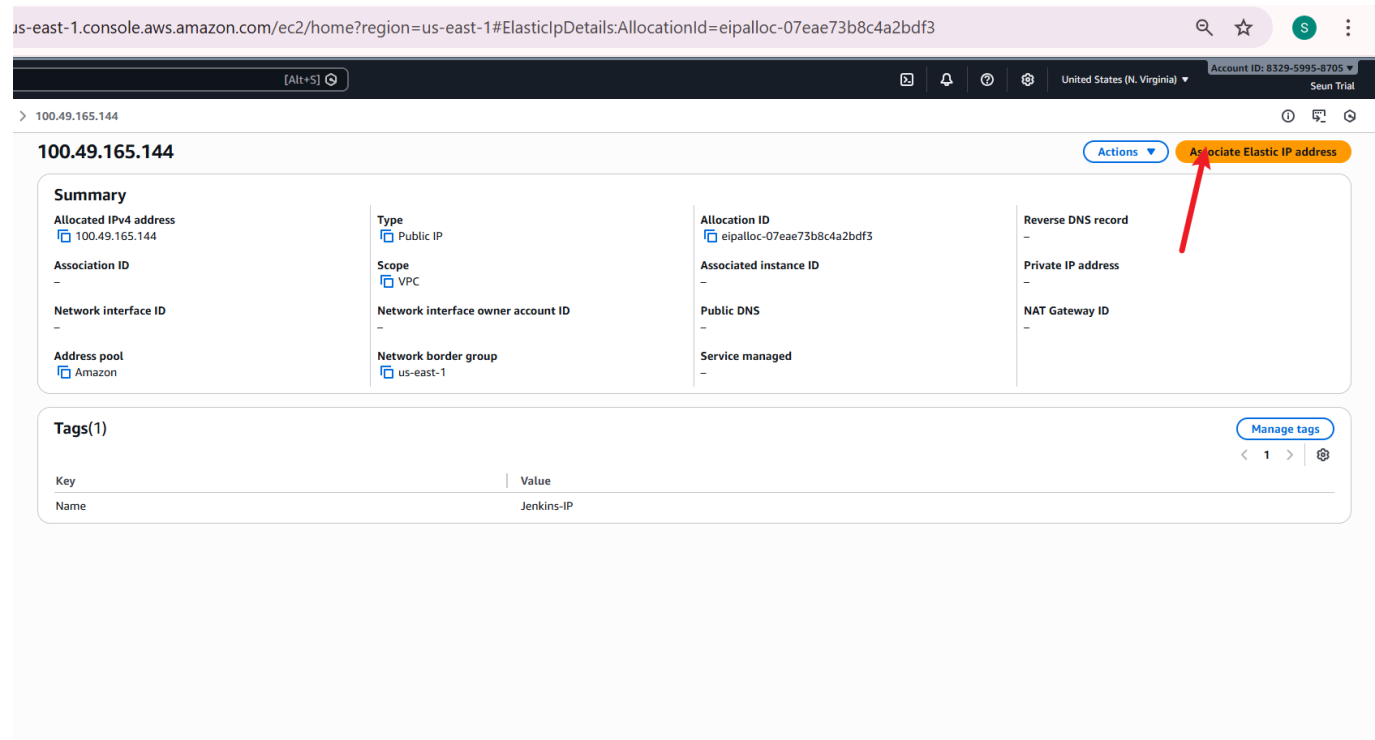
Click on the new IP in the list.



Step 28: Associate Elastic IP

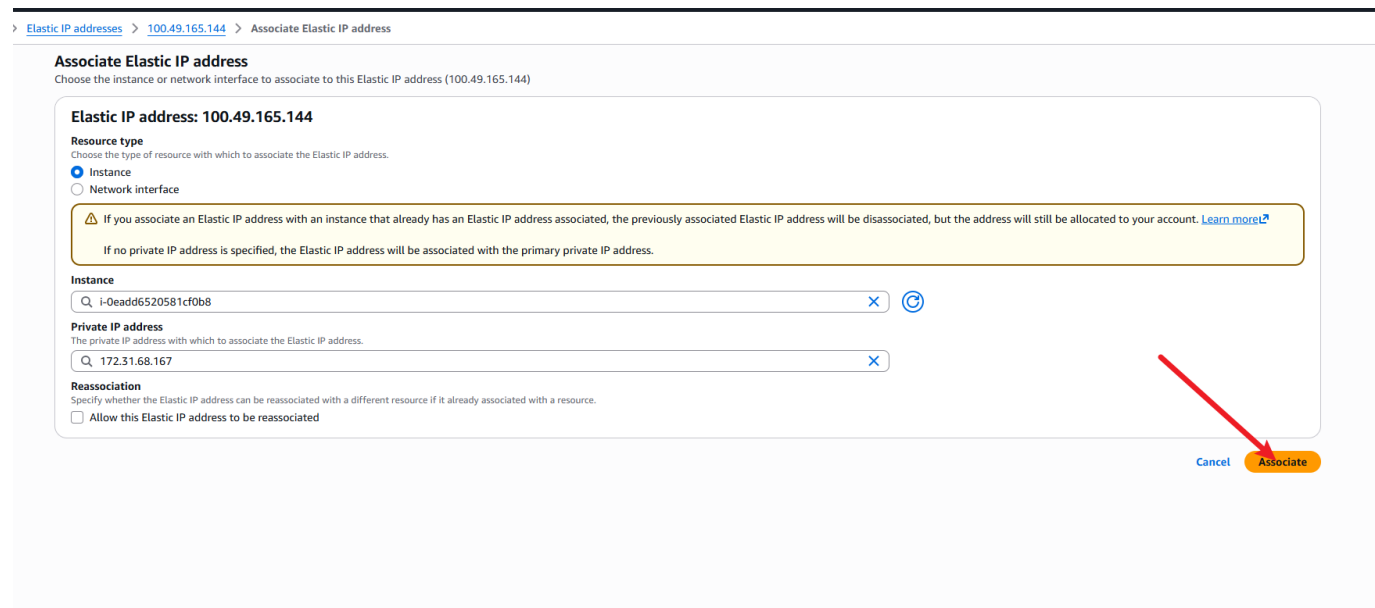
Link it to your EC2 instance.

- Click "Associate Elastic IP address".



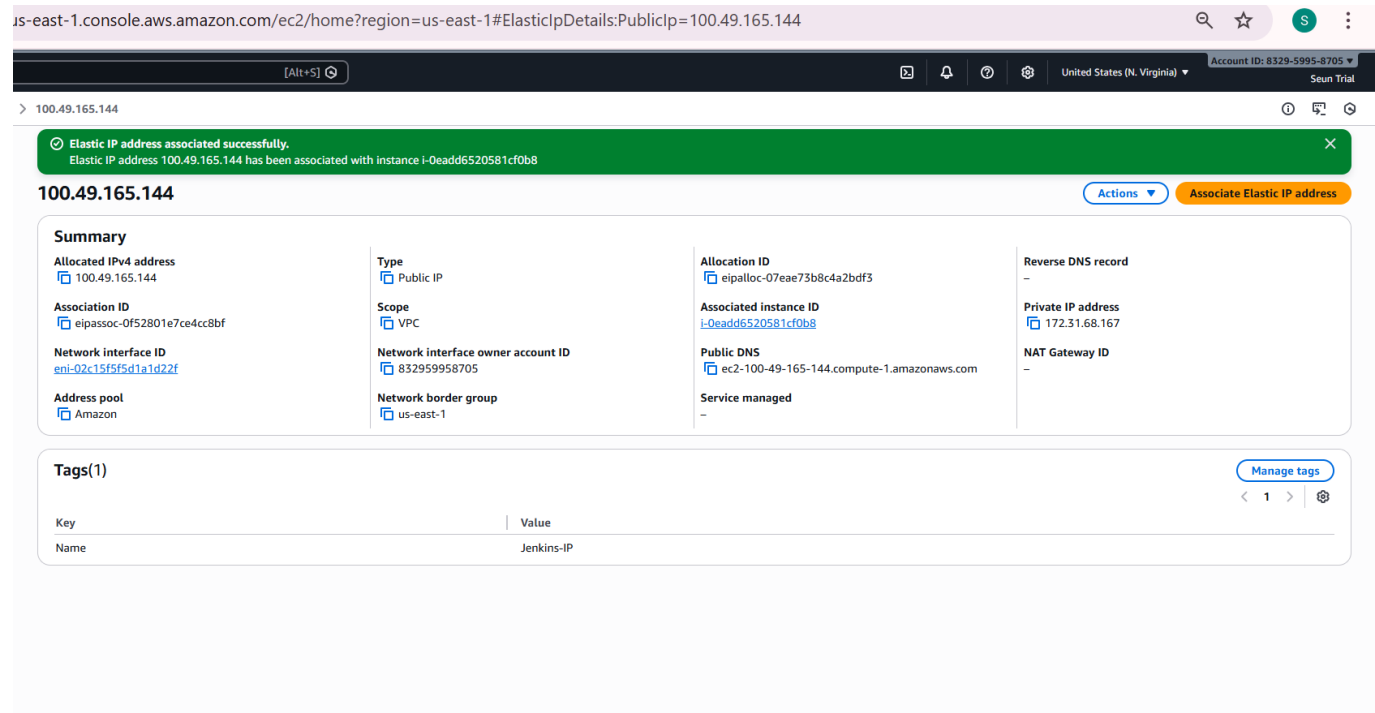
Step 29: Select Instance and Associate

Choose your instance and private IP, then associate.



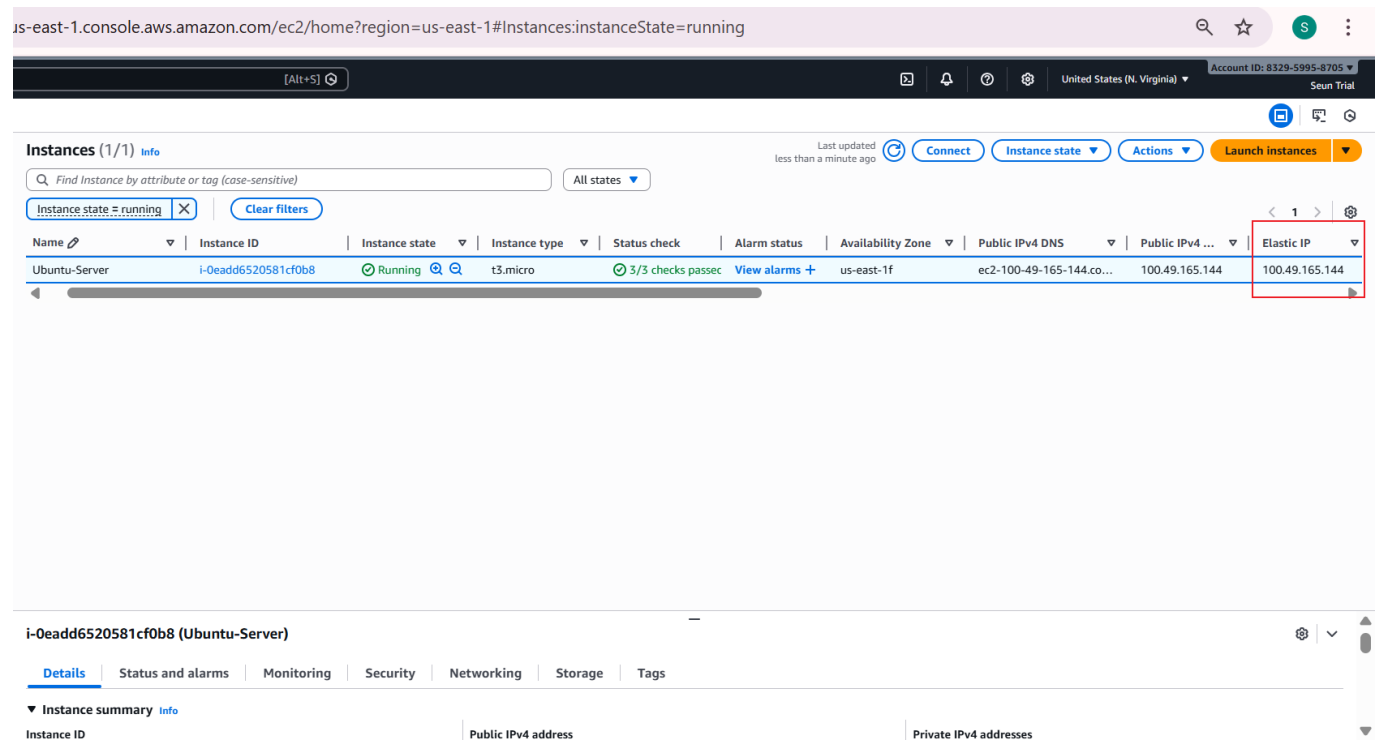
Step 30: Confirm Association

Verify the success message.



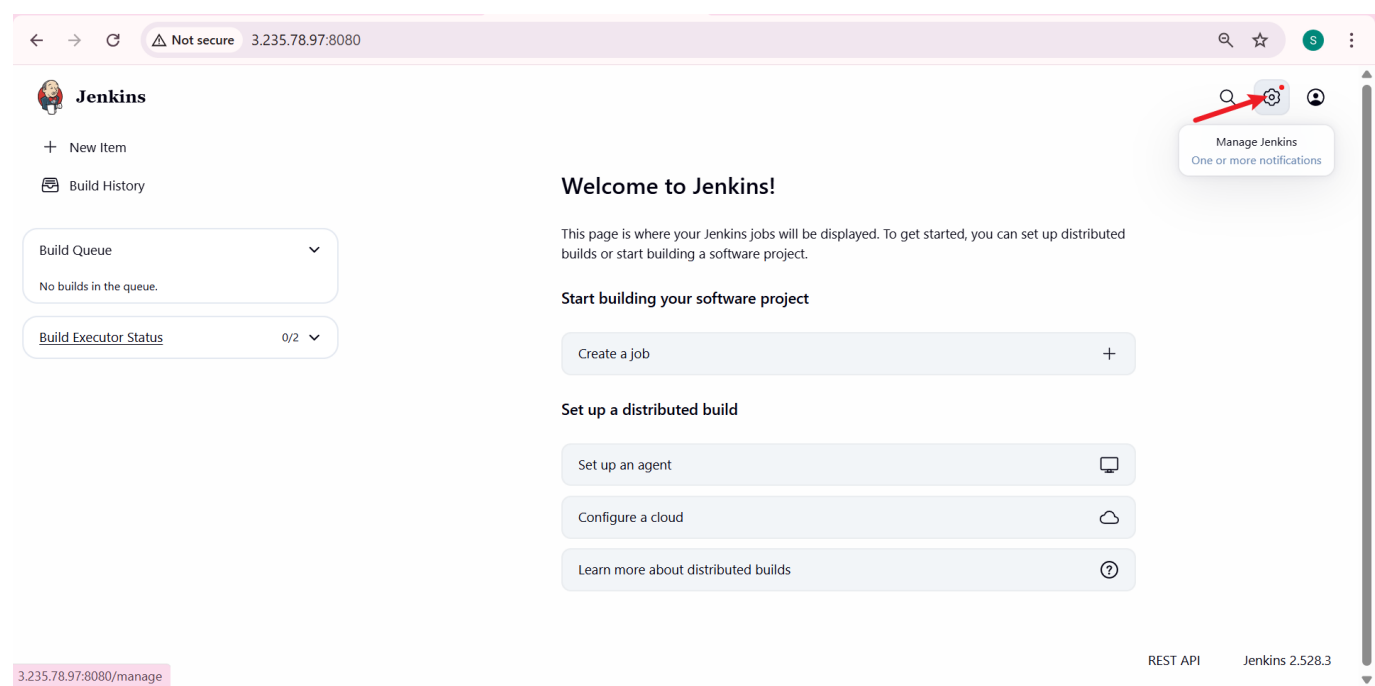
Step 31: Check Instance for Elastic IP

Back on the instance details, confirm the new public IP.



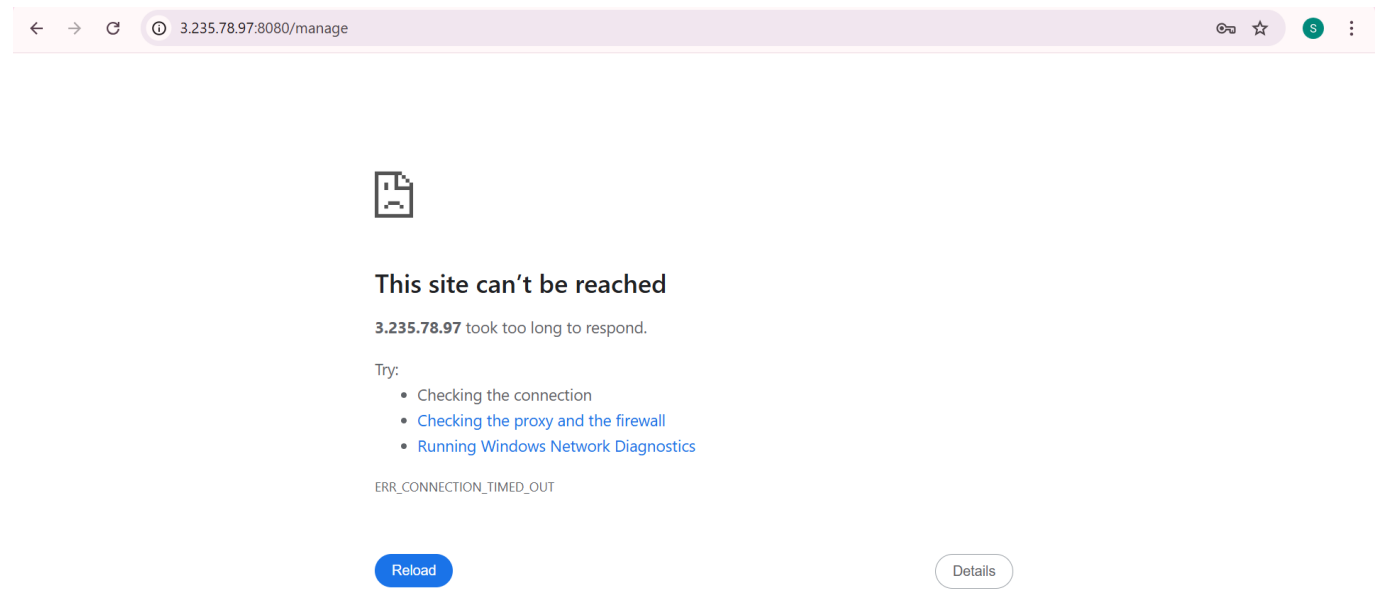
Step 32: Access Manage Jenkins

On the Jenkins dashboard, click "Manage Jenkins".



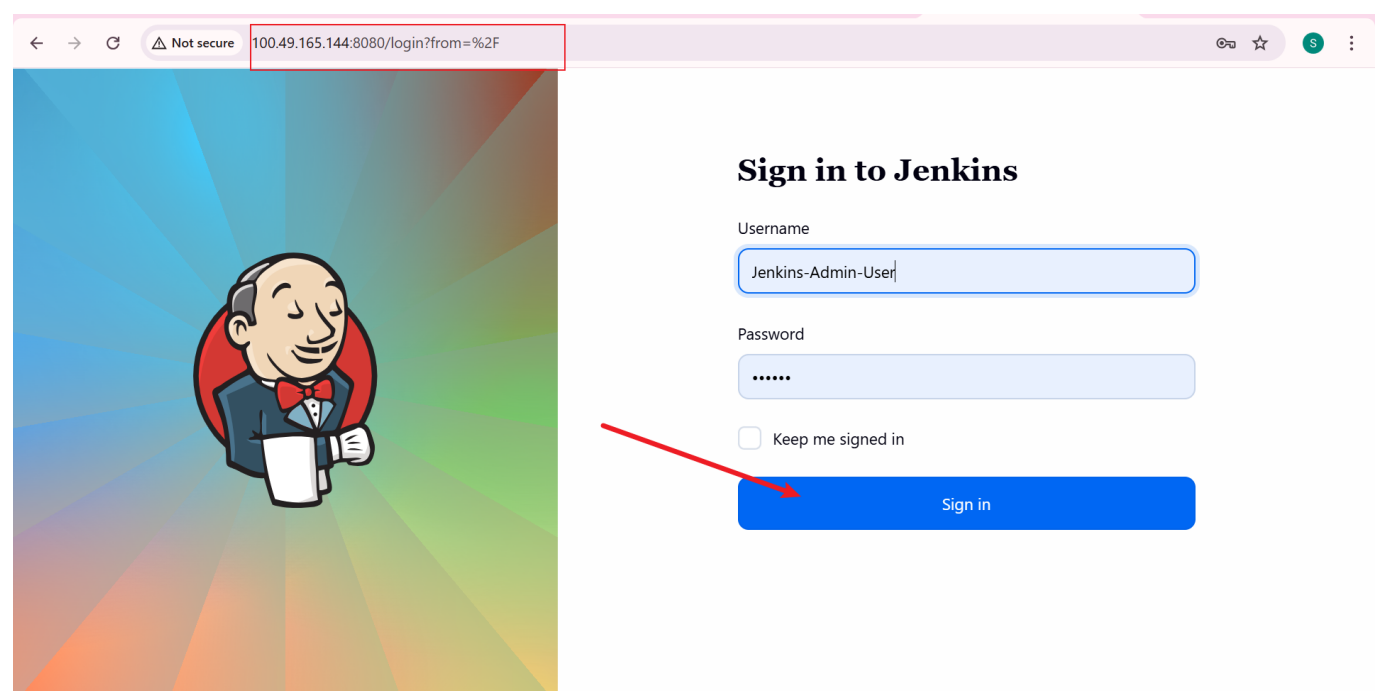
Step 33: Test Site with Elastic IP

If the site breaks, visit <http://<elastic-ip>:8080>.



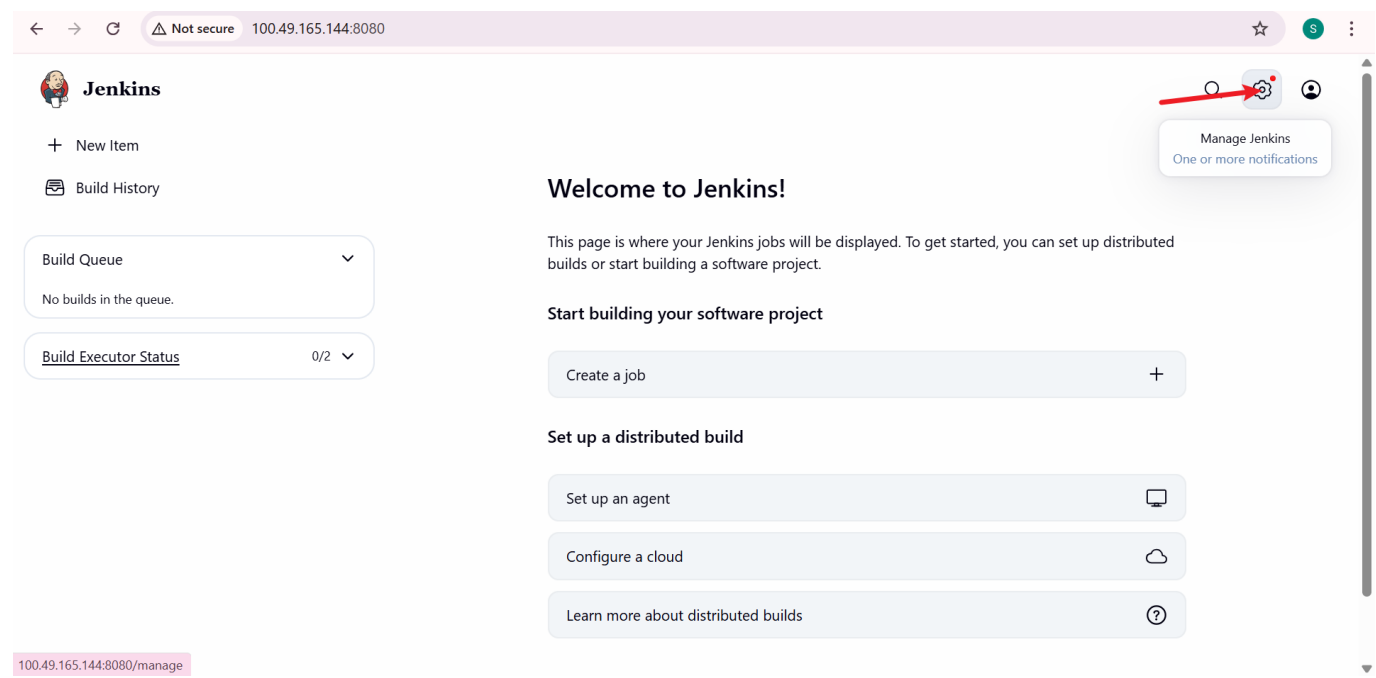
Step 34: Sign In with New IP

Jenkins now works on the Elastic IP; log in.



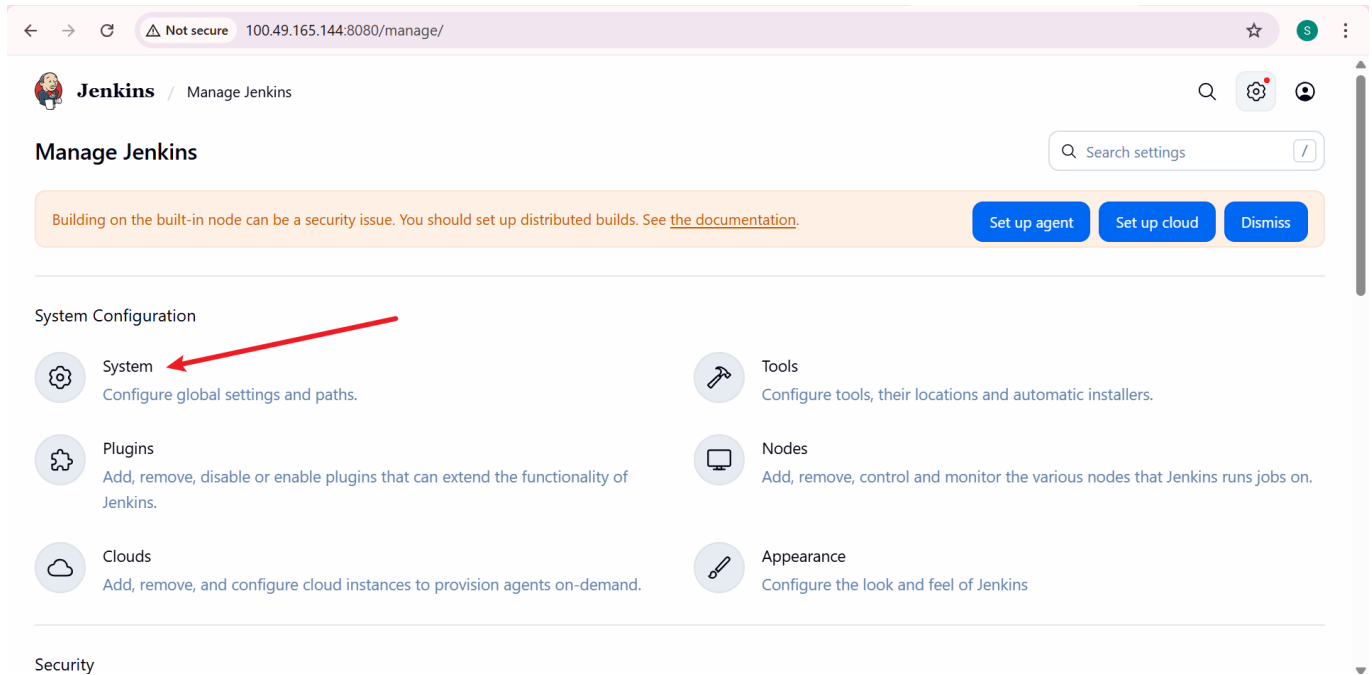
Step 35: Continue to Manage Jenkins

Proceed to configuration.



Step 36: Configure System Settings

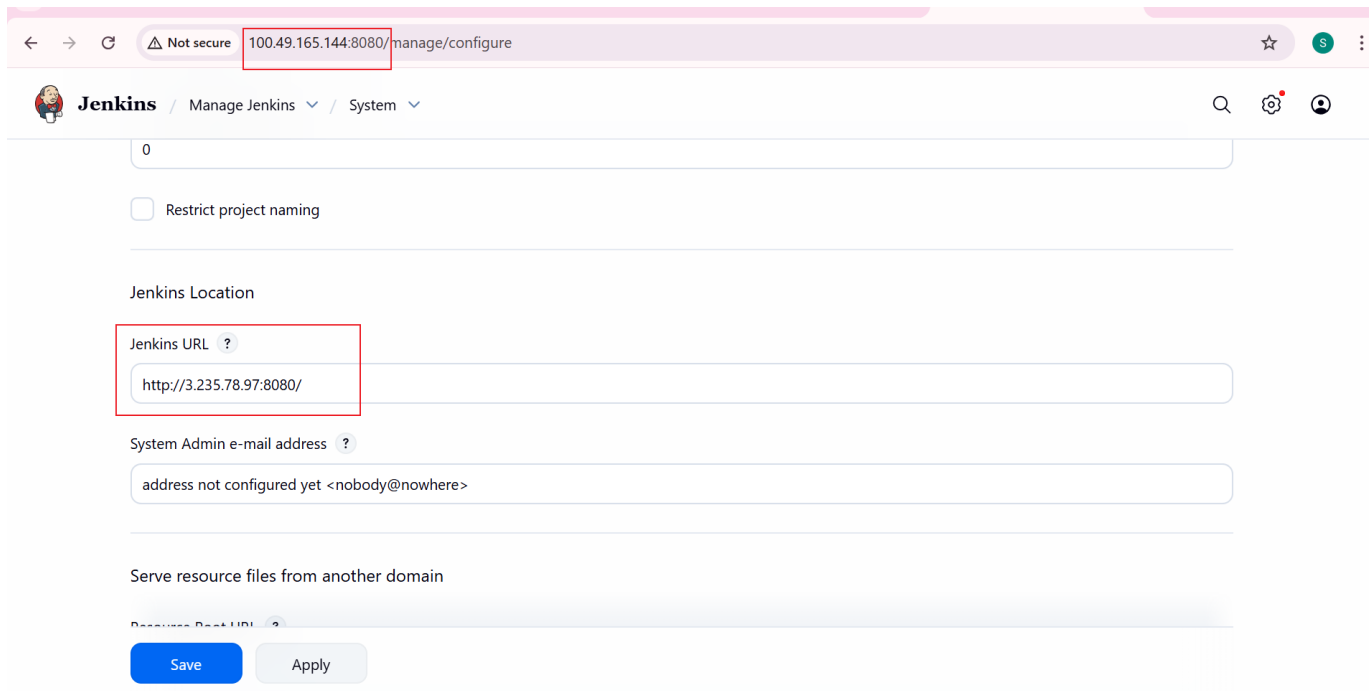
Click "Configure System" for global settings.



The screenshot shows the Jenkins 'Manage Jenkins' interface. The browser address bar displays '100.49.165.144:8080/manage/'. The page title is 'Manage Jenkins'. A search bar is present with the text 'Search settings'. A warning banner states: 'Building on the built-in node can be a security issue. You should set up distributed builds. See the documentation.' Below this, there are three buttons: 'Set up agent', 'Set up cloud', and 'Dismiss'. The 'System Configuration' section is visible, with a red arrow pointing to the 'System' option. The 'System' option is described as 'Configure global settings and paths.' Other options include 'Tools', 'Nodes', 'Appearance', 'Plugins', and 'Clouds'. The 'Security' section is partially visible at the bottom.

Step 37: Scroll to Jenkins URL

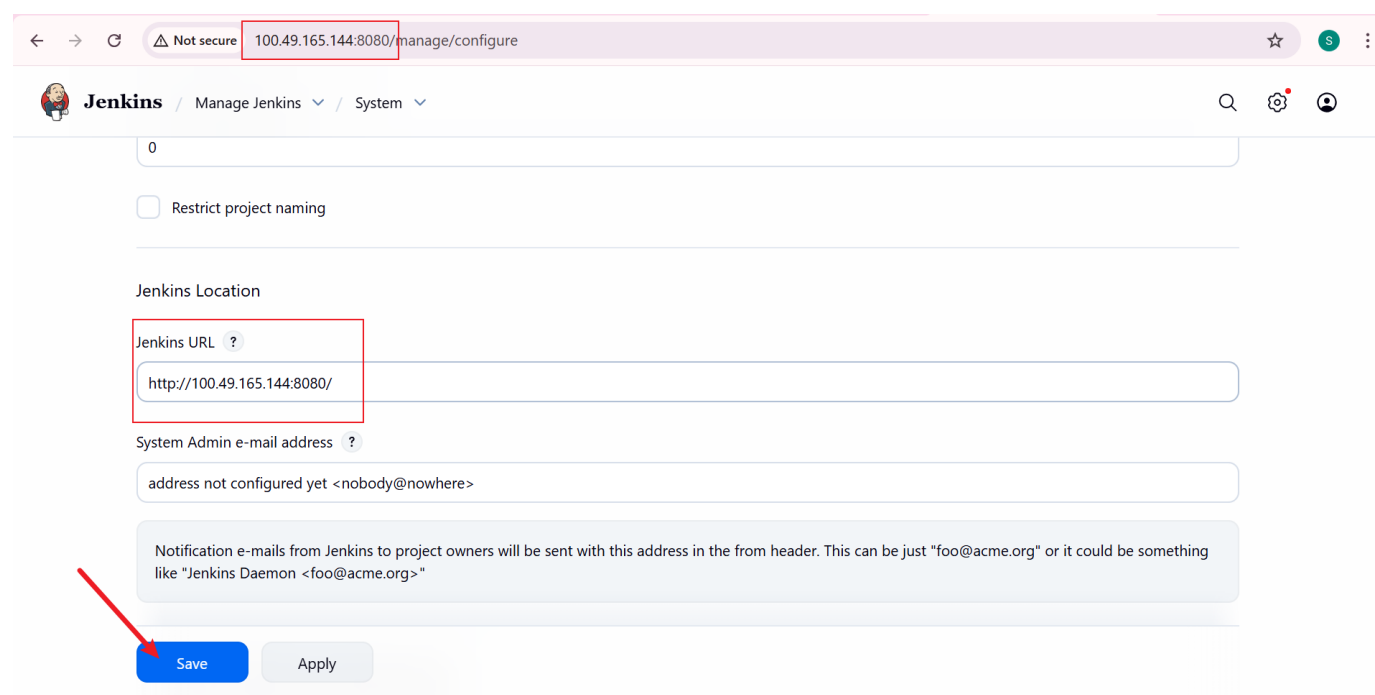
Find the "Jenkins Location" section.



The screenshot shows the Jenkins 'System' configuration page. The browser address bar displays '100.49.165.144:8080/manage/configure'. The page title is 'Jenkins / Manage Jenkins / System'. The 'Jenkins Location' section is highlighted with a red box. It contains a 'Jenkins URL' field with a question mark icon, which currently contains the value 'http://3.235.78.97:8080/'. Below this is a 'System Admin e-mail address' field with a question mark icon, containing the text 'address not configured yet <nobody@nowhere>'. At the bottom, there are 'Save' and 'Apply' buttons.

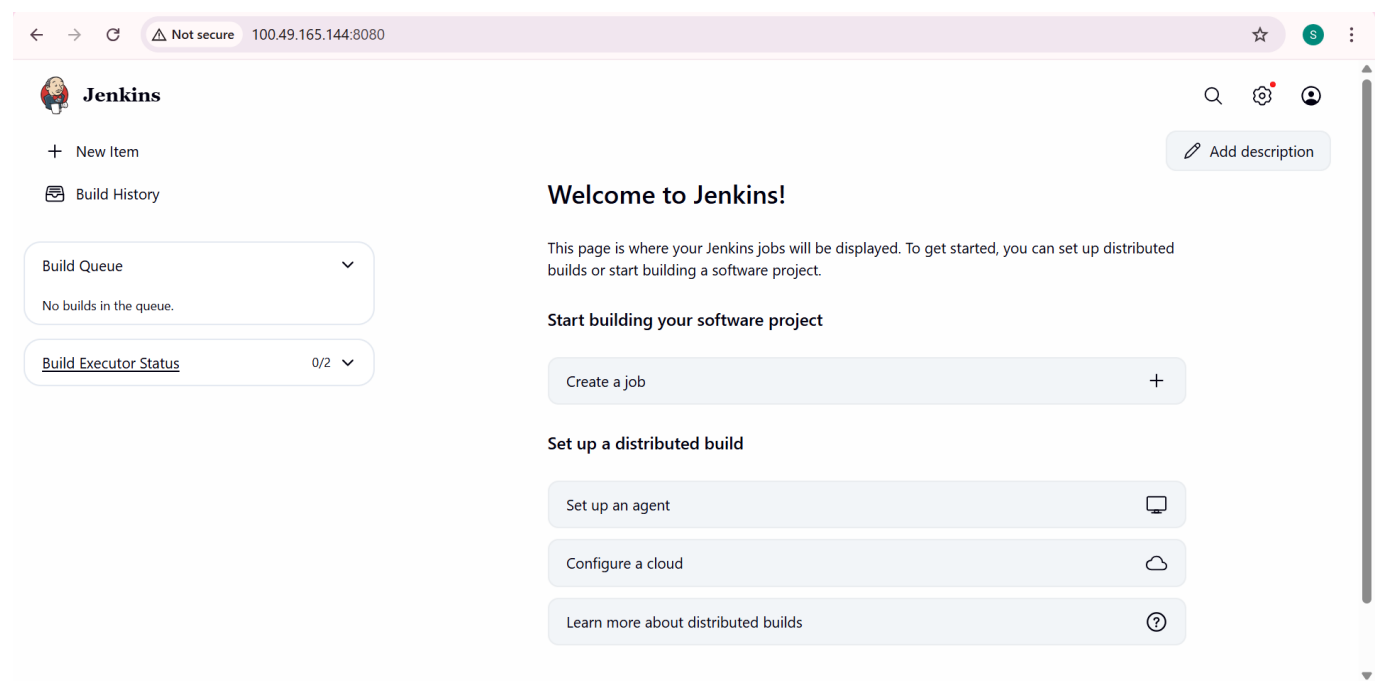
Step 38: Update Jenkins URL

Change to `http://<elastic-ip>:8080` and save.



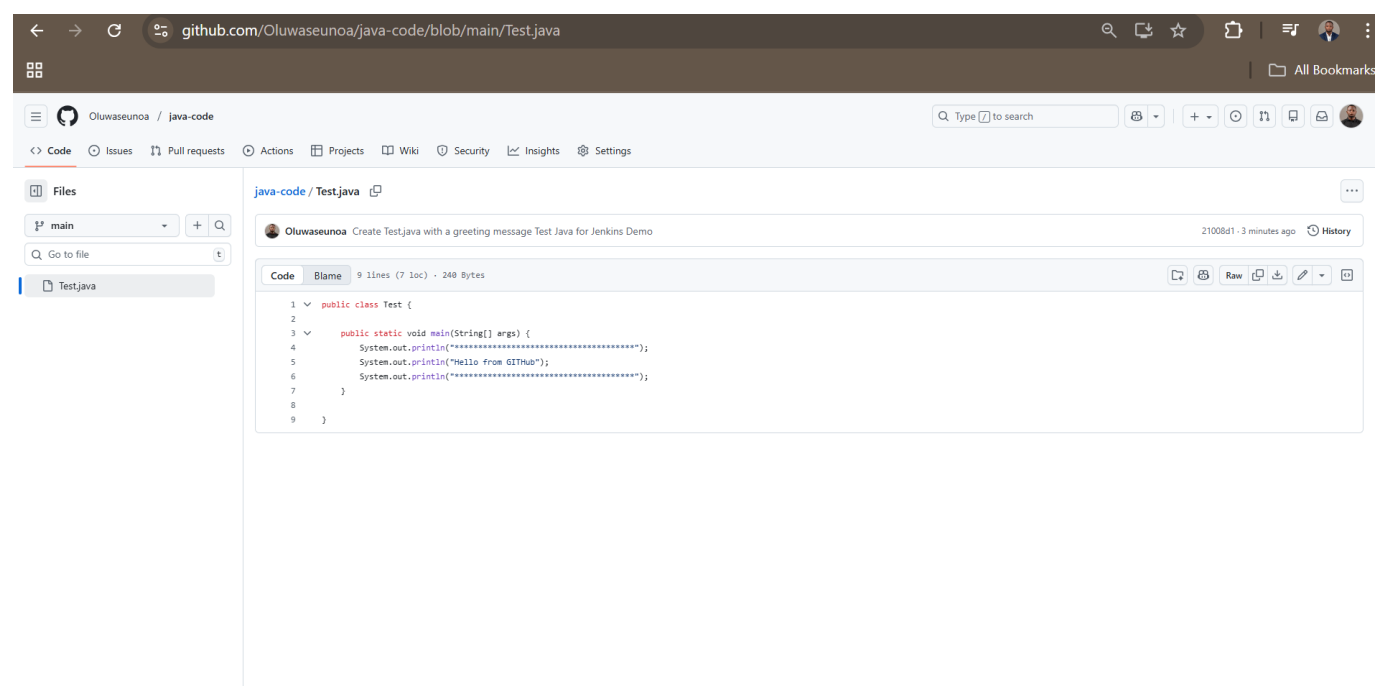
Step 39: Reload to Verify

Refresh the page to ensure accessibility.



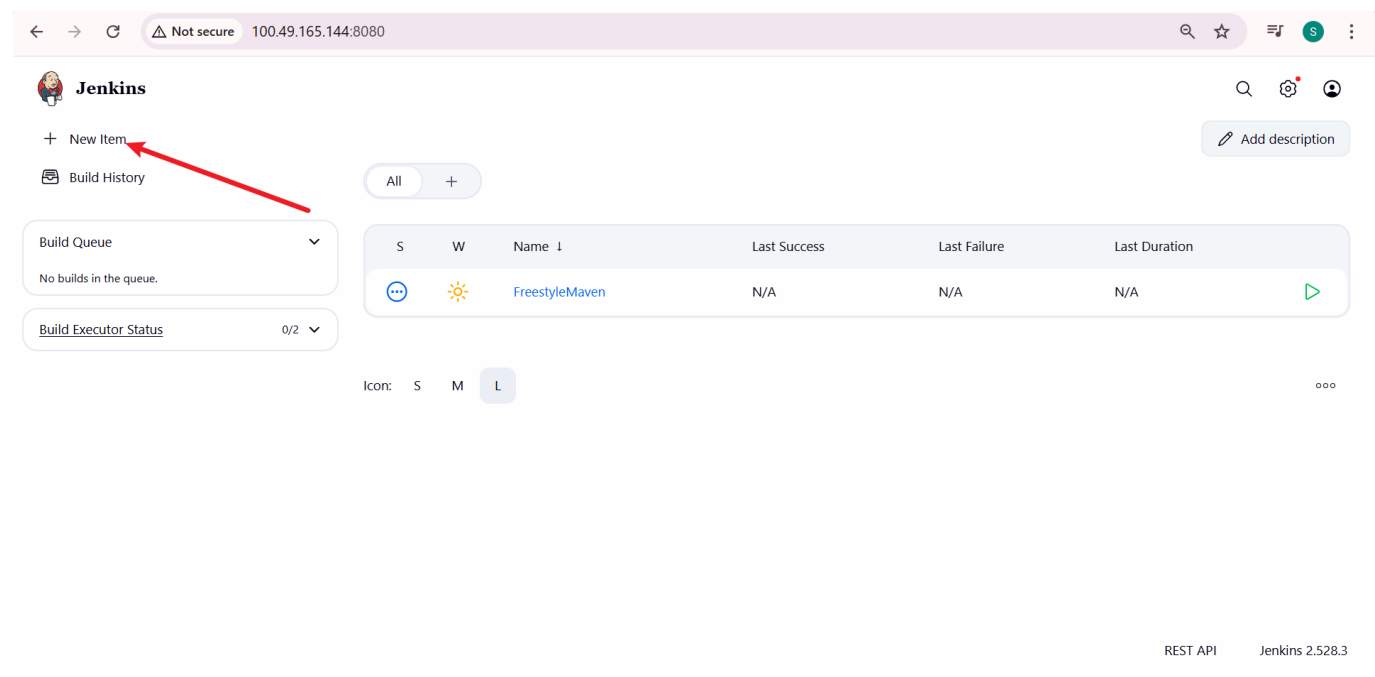
Step 40: Prepare Your GitHub Repository

Use a repository like "java-code" with **Test.java** (simple greeting code).



Step 41: Create New Jenkins Job

On dashboard, click "New Item".



Step 42: Name and Select Freestyle Project

Name it (e.g., "JavaBuild"), select Freestyle, OK.

← → ↻ Not secure 100.49.165.144:8080/view/all/newJob

Jenkins / All / New Item

New Item

Enter an item name

Java-Project-Git

Select an item type

- Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- Pipeline**
Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.
- Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
- Folder**
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a

OK

Step 43: Configure Source Code Management

Scroll to SCM, select Git.

← → ↻ Not secure 100.49.165.144:8080/job/Java-Project-Git/configure

Jenkins / Java-Project-Git / Configuration

Configure

- General
- Source Code Management
- Triggers
- Environment
- Build Steps
- Post-build Actions

Source Code Management

Connect and manage your code repository to automatically pull the latest code for your builds.

☐ None

☒ Git ?

Repositories ?

Repository URL ?

Please enter Git repository.

Credentials ?

- none -

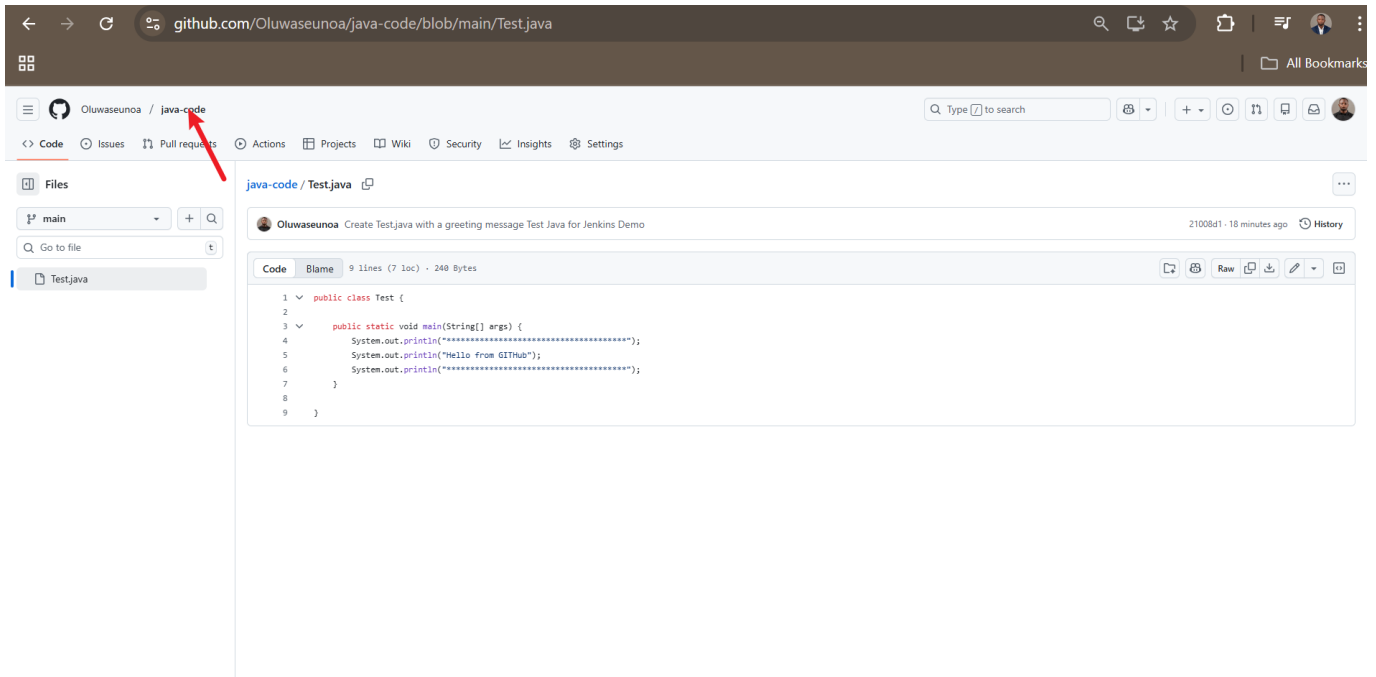
+ Add

Advanced

Save Apply

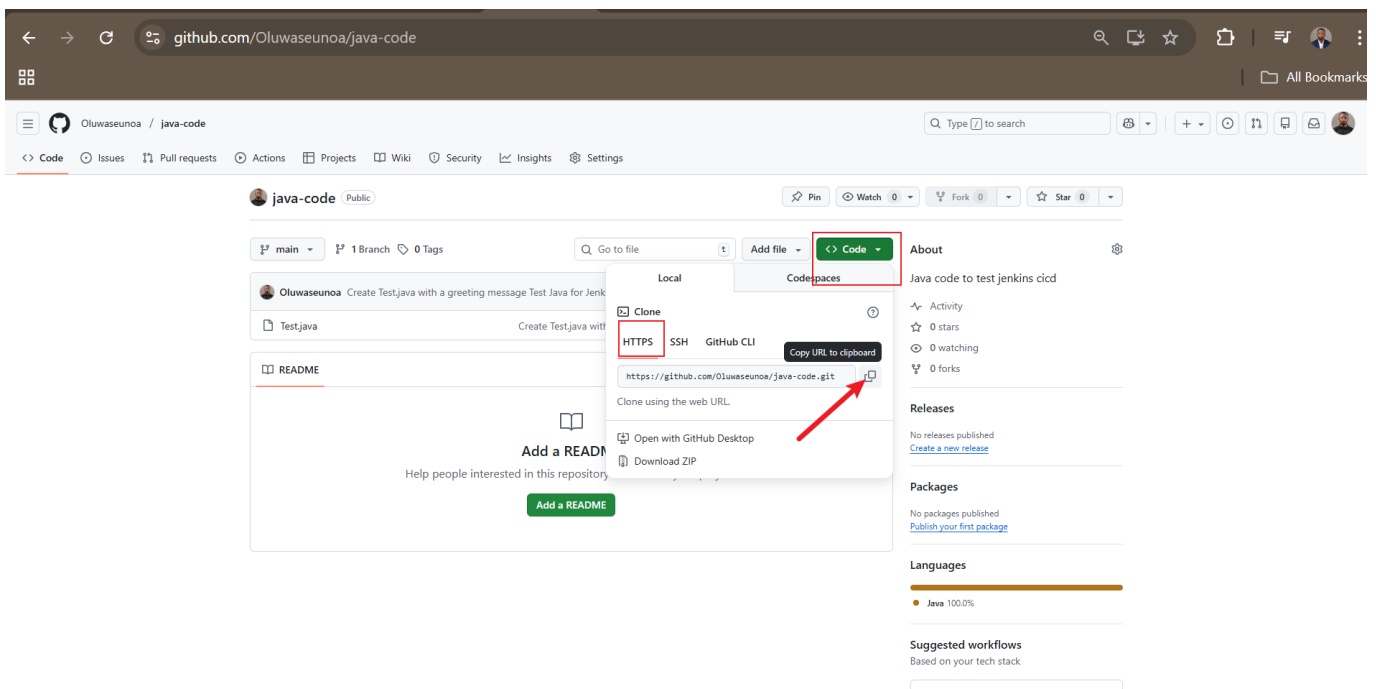
Step 44: Go to Your GitHub Repository

Open your "java-code" repo on GitHub.



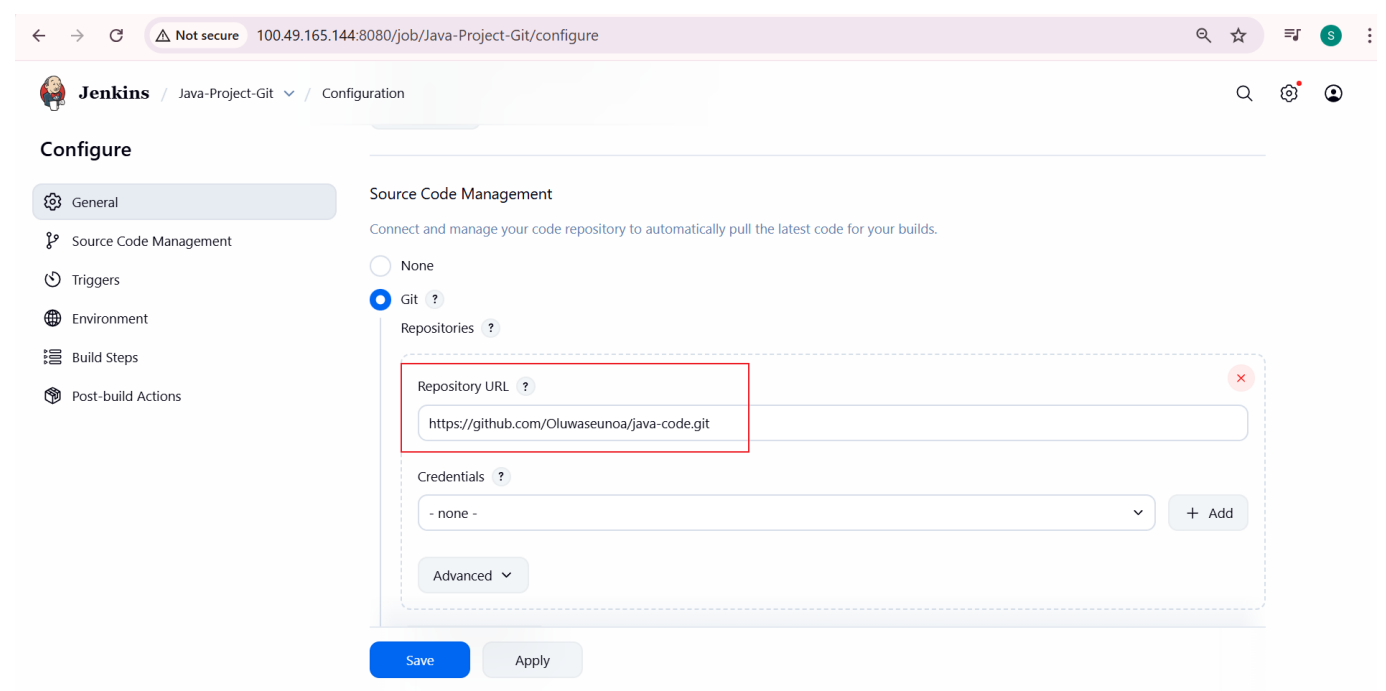
Step 45: Copy Repository URL

Click "Code" and copy the HTTPS URL.



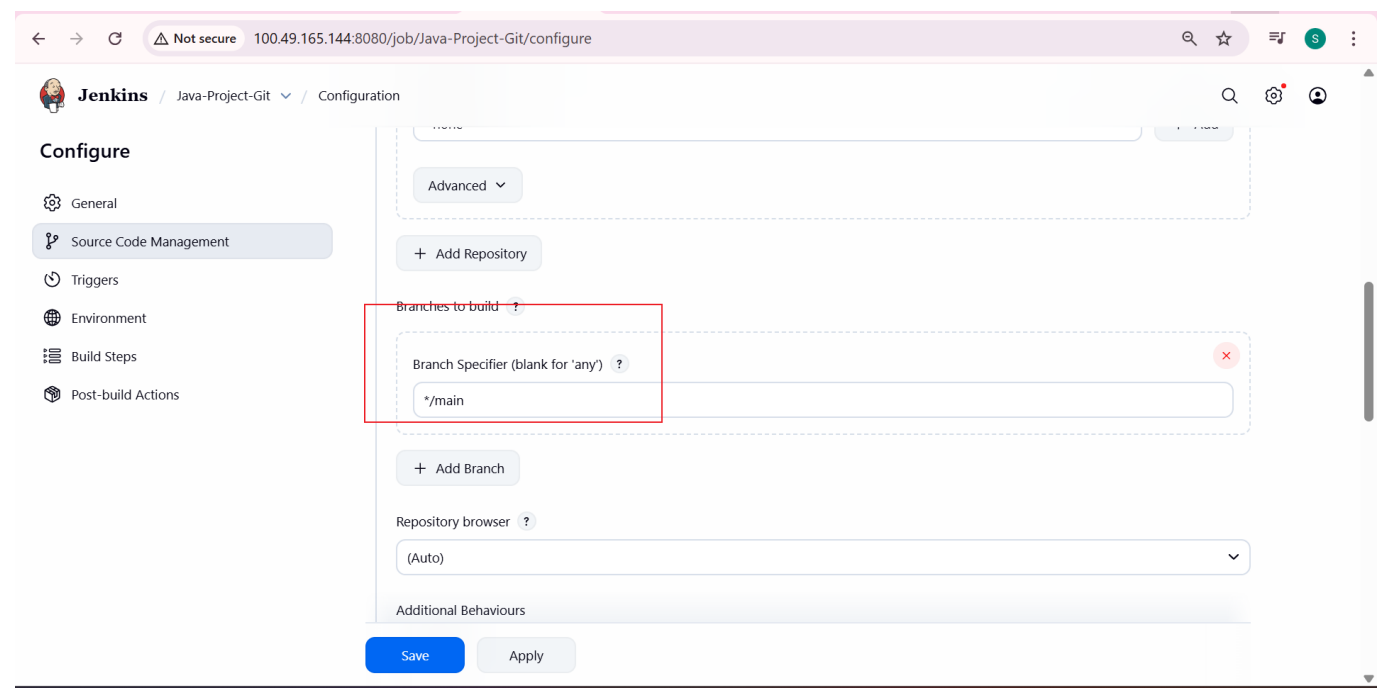
Step 46: Paste Repository URL in Jenkins

In Jenkins, paste under "Repository URL", then scroll.



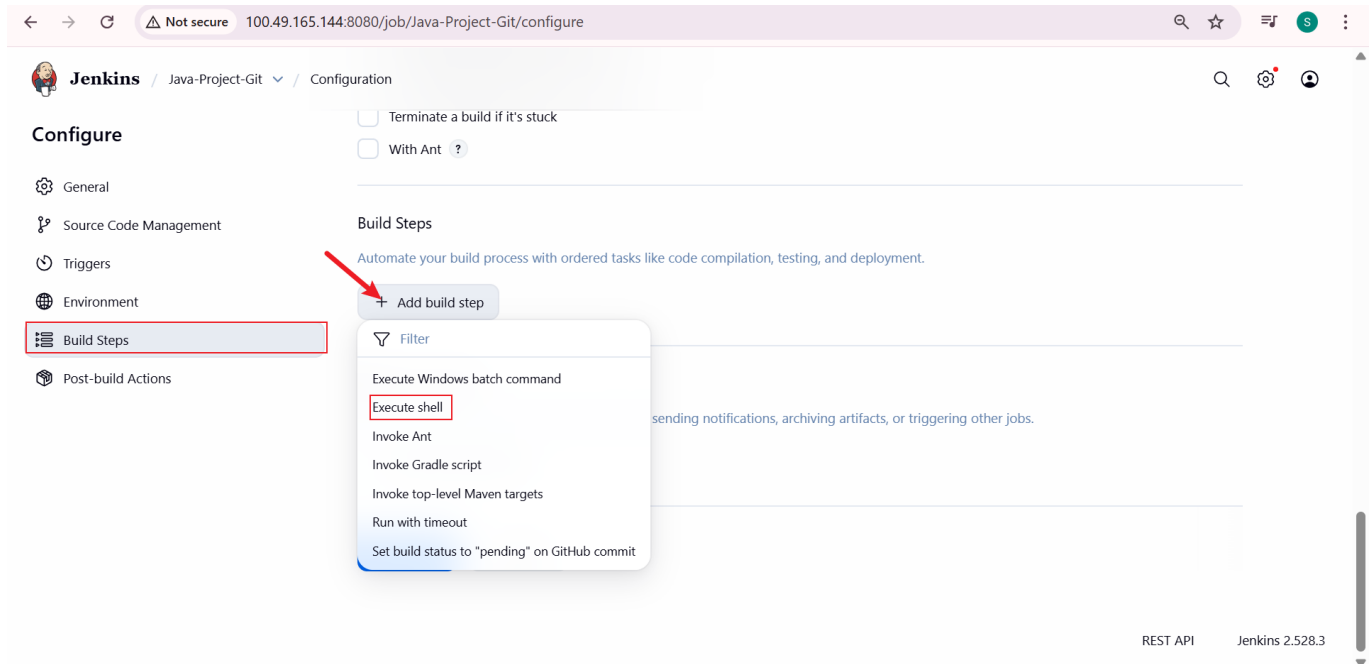
Step 47: Specify Branch to Build

Under Branches, set to "main" (or your branch).



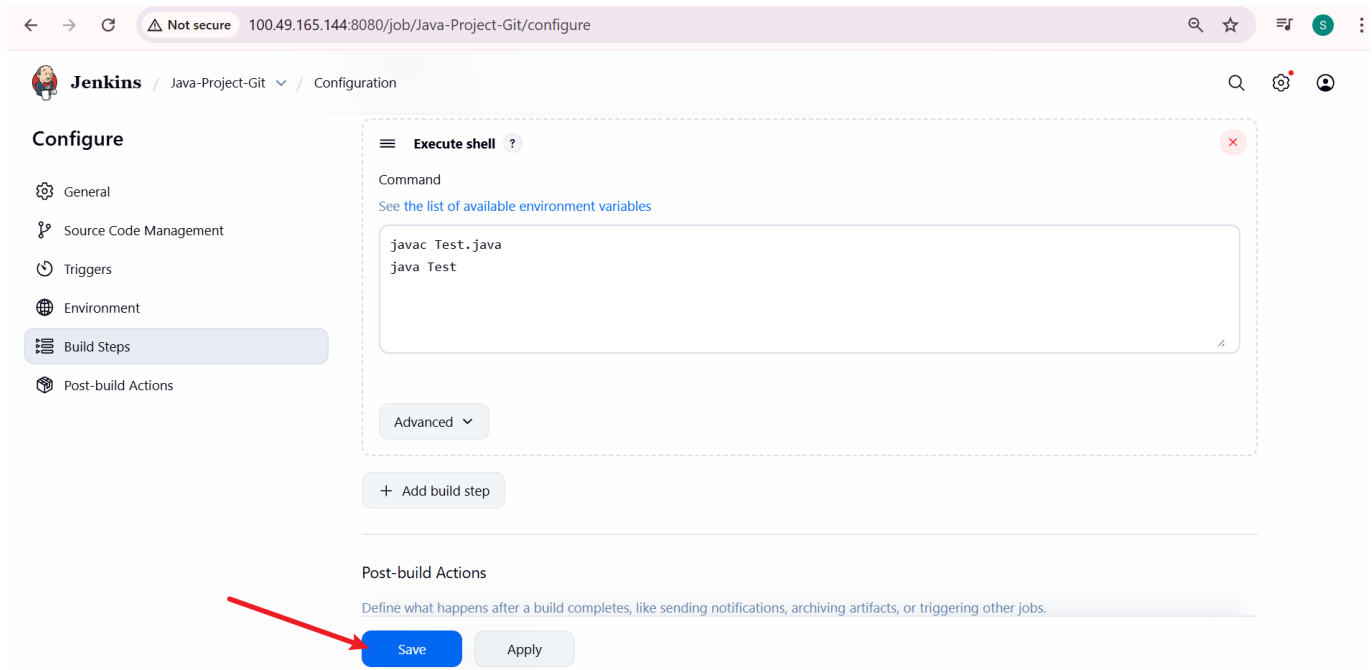
Step 48: Add Build Step

Under Build Steps, add "Execute shell".



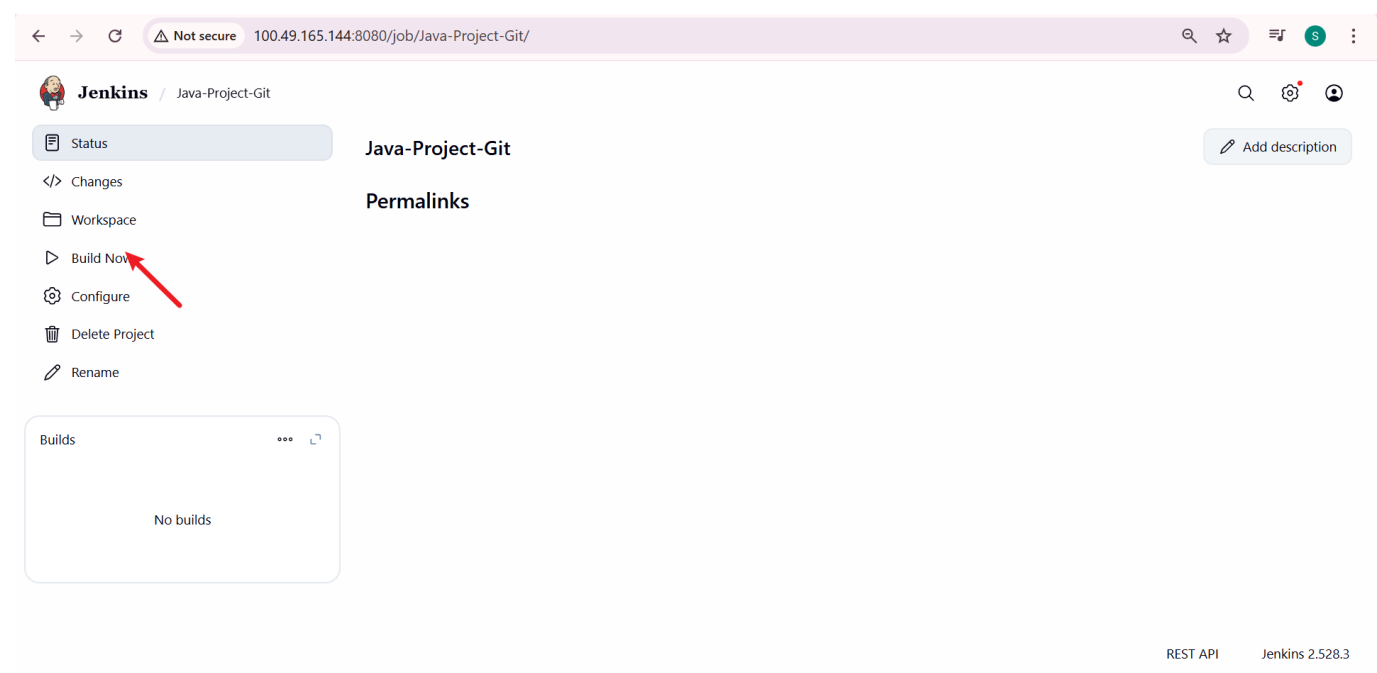
Step 49: Enter Build Command

Command: `javac Test.java` (compiles your Java file). Save.



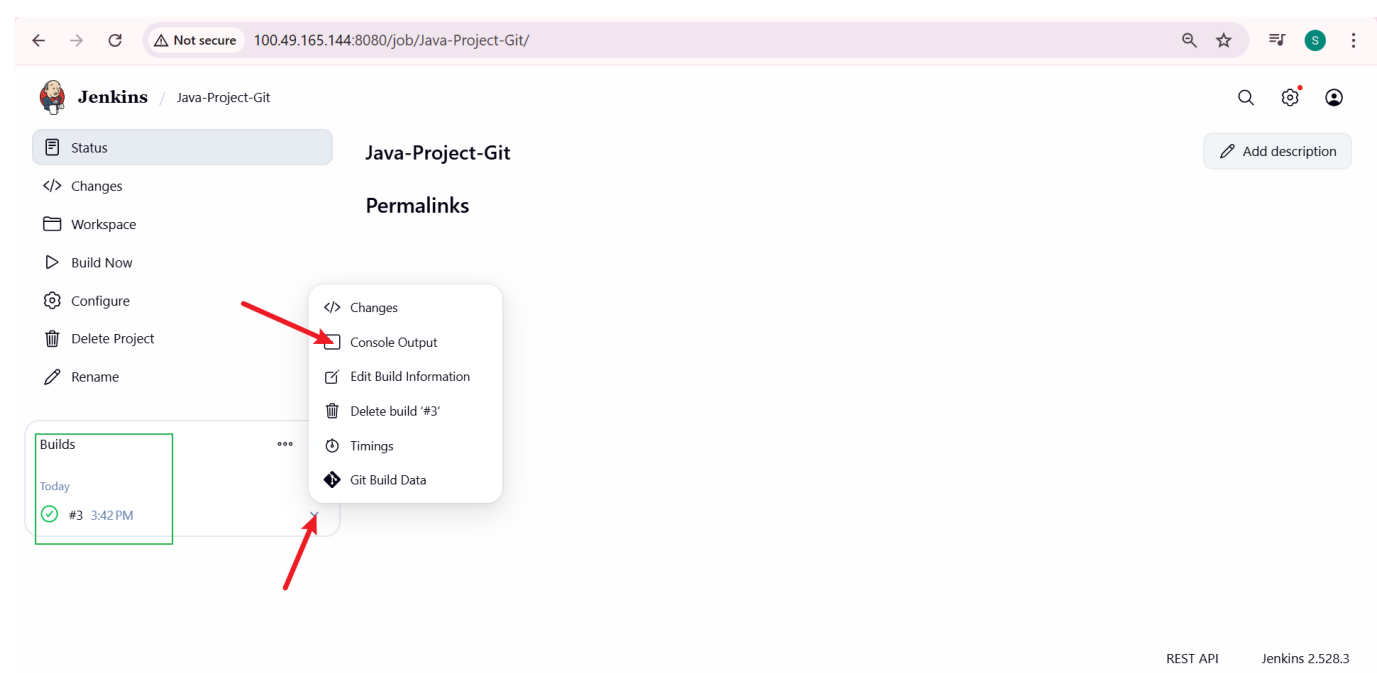
Step 50: Trigger the Build

Click "Build Now" to run the job.



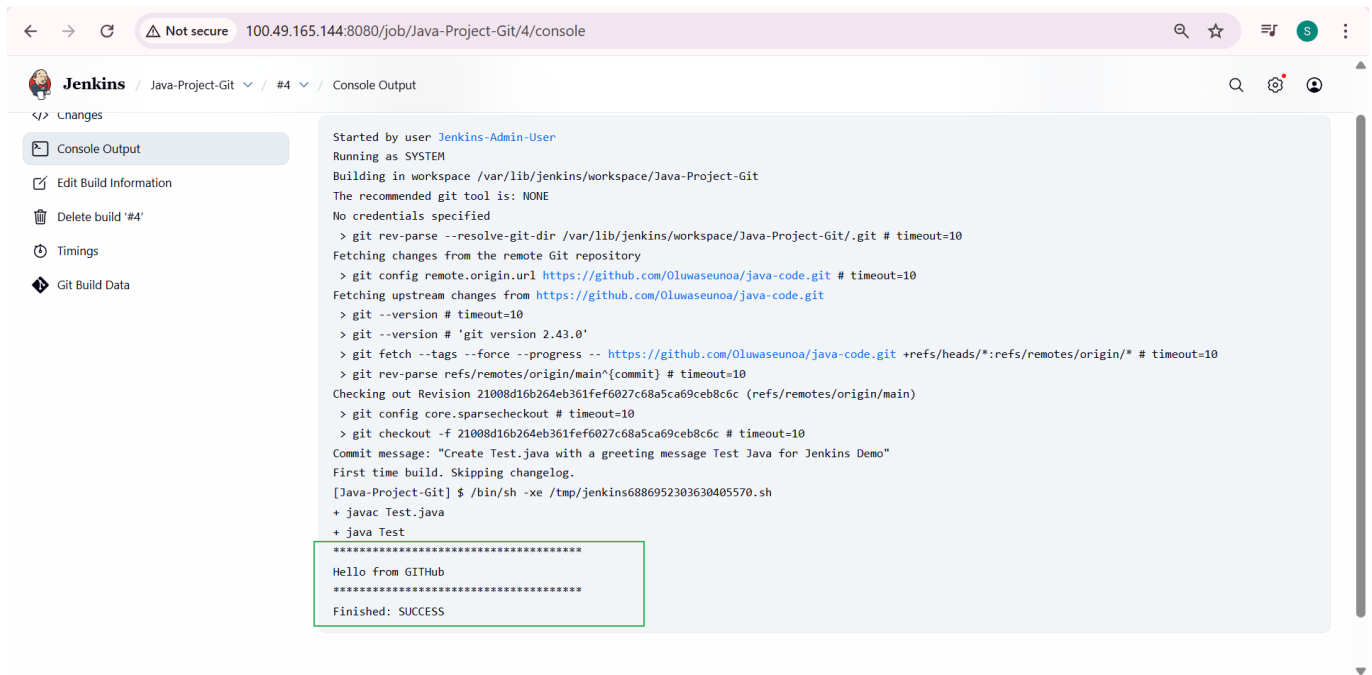
Step 51: Check Build Status

View build history; select Console Output for details.



Step 52: Verify Output

Console shows compilation success and any output (e.g., greeting if running `java Test`—add if needed).



The screenshot shows the Jenkins web interface for a build named 'Java-Project-Git' with build number '#4'. The console output displays the following steps:

```
Started by user Jenkins-Admin-User
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/Java-Project-Git
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/Java-Project-Git/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/Oluwaseunola/java-code.git # timeout=10
Fetching upstream changes from https://github.com/Oluwaseunola/java-code.git
> git --version # timeout=10
> git --version # 'git version 2.43.0'
> git fetch --tags --force --progress -- https://github.com/Oluwaseunola/java-code.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 21008d16b264eb361fef6027c68a5ca69ceb8c6c (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 21008d16b264eb361fef6027c68a5ca69ceb8c6c # timeout=10
Commit message: "Create Test.java with a greeting message Test Java for Jenkins Demo"
First time build. Skipping changelog.
[Java-Project-Git] $ /bin/sh -xe /tmp/jenkins6886952303630405570.sh
+ javac Test.java
+ java Test
*****
Hello from GitHub
*****
Finished: SUCCESS
```

Next Steps and Best Practices

- **Add Execution:** Modify the shell step to `javac Test.java && java Test` to run and print the greeting.
- **Triggers:** Add GitHub webhook for automatic builds on push (install GitHub plugin if needed).
- **Security:** Set up HTTPS (self-signed or with domain), user roles via plugins.
- **Scaling:** Use pipelines for complex workflows; integrate Docker for containerized builds.
- **Cleanup:** Release Elastic IP when done to avoid charges.
- **Troubleshooting:** Check Jenkins logs (`/var/log/jenkins/jenkins.log`), ensure Java path is in `$PATH`.